HIGH TEEN DAY · PERFORMANCE STUDY

하드웨어 자원과 성능 병목을 읽는 법

현재 수집 중인 165개 지표와 수집 후보를 바탕으로,
숫자에서 병목 가설·검증 실험까지 스스로 연결하는 개인 학습서

대상 스택

Spring Boot 3.4 · Java 17 · MySQL 8 · Redis · Prometheus · k6

작성 기준일 2026-08-18

이 책의 목적

> **한 문장 목표**
>
> 대시보드에서 숫자 하나가 튀었을 때 곧바로 원인을 단정하지 않고, 그 숫자가 뜻하는 물리적 현상·함께 봐야 할 지표·반증 방법·다음 실험을 스스로 설명할 수 있게 되는 것.

이 문서는 지표 수집을 자동화하거나 시스템을 다시 만드는 설계서가 아니다. 이미 수집되는 지표와 앞으로 수집하면 좋은 지표를 **학습 재료**로 삼아, CPU·메모리·디스크·네트워크·커넥션 풀·JVM·데이터베이스가 서로 어떻게 영향을 주는지 이해하는 데만 초점을 둔다.

각 장은 같은 질문 순서를 반복한다. 이 순서를 습관으로 만들면 지표 이름이 달라져도 분석 방식은 그대로 남는다.

순서	스스로 물을 질문	목적
1. 정의	무엇을, 어느 범위에서, 어떤 단위로 센 값인가?	숫자의 정체 확인
2. 방향	높거나 낮으면 어떤 물리적 현상이 생기는가?	현상 해석
3. 동반	같이 움직여야 하는 상관 지표는 무엇인가?	가설 강화
4. 반증	이 값만 움직이고 다른 값은 그대로면 무엇을 배제하는가?	오진 방지
5. 귀속	어느 URI·SQL·Redis 명령·스케줄러가 비용을 만들었나?	코드 후보 좁히기
6. 실험	조건 하나만 바꿔 원인 가설을 깨거나 지지할 수 있는가?	인과 확인

이 책을 읽는 권장 순서

1. 1~3장에서 지표 언어와 전체 자원 흐름을 익힌다.
2. 현재 의심하는 자원의 장을 골라 CPU → 대기 → 하위 자원 순으로 읽는다.
3. 14장의 현재 카탈로그 사전에서 실제 키를 찾아 정의와 동반 지표를 확인한다.
4. 16장의 실전 문제를 먼저 풀고, 17장의 해설에서 추론 과정까지 비교한다.

> **중요: 이 책의 숫자는 세 종류다**
>
> **관측 사실**은 현재 프로젝트나 실행 기록에서 확인된 값이다. **현재 정책**은 도구가 WARN/FAIL을 판단하도록 정한 기준이다. **예시**는 개념 설명을 위한 가상 값이다. 서로 섞지 않는다.

목차

이 책의 목적 2

이 책을 읽는 권장 순서 . . . 2

목차 3

1. 병목 분석의 기본 언어 7

1.1 다섯 개의 핵심 질문 . . . 7

1.2 자원 부족과 비효율은 다르다 . . . 7

1.3 관측 사실 → 가설 → 증명 . . . 8

2. Prometheus 지표를 읽는 법 9

2.1 Gauge, Counter, Histogram . . . 9

2.2 avg, max, p95가 서로 답하는 질문 . . . 9

2.3 p95는 무엇의 95분위인가 . . . 9

2.4 분자와 분모의 범위를 맞춘다 . . . 9

2.5 결측은 0이 아니다 . . . 9

3. CPU: 바쁨, 포화, 제한, 효율 10

3.1 CPU core와 percent . . . 10

3.2 throttling은 'CPU 사용률이 낮아도' 생길 수 있다 . . . 10

3.3 host CPU와 container CPU를 분리한다 . . . 10

3.4 CPU를 코드에 귀속한다 . . . 11

4. 메모리: heap, RSS, working set, cache 12

4.1 같은 메모리를 다른 층에서 본다 . . . 12

4.2 누수와 캐시를 시간 모양으로 구분한다 . . . 12

4.3 MySQL과 Redis 메모리 . . . 12

4.4 메모리 분석 순서 . . . 12

5. 디스크와 I/O: 바쁜 디스크와 느린 디스크 13

5.1 bytes/sec와 latency는 다른 질문 . . . 13

5.2 MySQL 쓰기 경로의 실제 비용 . . . 13

5.3 임시 디스크 테이블 . . . 13

6. MySQL: 쿼리, 락, 버퍼, 커넥션 14

6.1 MySQL 병목을 네 갈래로 나눈다 . . . 14

6.2 QPS/RPS는 N+1의 경보다 . . . 14

6.3 Buffer pool hit ratio를 과신하지 않는다 . . . 14

6.4 스캔과 정렬 . . . 14

6.5 락 지표의 시간 의미 . . . 14

6.6 커넥션 수와 실행 쿼리 수 . . . 15

7. 네트워크와 k6 시간 분해 16
7.1 클라이언트 duration을 조각낸다 . . . 16
7.2 bytes/sec와 packet error . . . 16
7.3 TCP accept queue와 TIME_WAIT . . . 16
7.4 load generator도 측정 대상이다 . . . 16
8. Tomcat과 Hikari: 두 개의 queue 17
8.1 요청은 두 번 줄을 설 수 있다 . . . 17
8.2 현재 프로젝트의 구조적 불균형 . . . 17
8.3 acquire와 usage가 답하는 질문 . . . 17
8.4 풀 확장보다 transaction diet를 먼저 본다 . . . 17
9. Redis: hit ratio, 명령 비용, eviction 19
9.1 Redis는 빠르다는 전제를 버린다 . . . 19
9.2 hit ratio의 해석 . . . 19
9.3 eviction과 expiration . . . 19
9.4 프로젝트 사례: KEYS . . . 19
10. JVM과 GC: 할당, 생존, 정지 20
10.1 GC를 세 질문으로 나눈다 . . . 20
10.2 pause max와 HTTP p99의 관계 . . . 20
10.3 Young와 Full을 분리한다 . . . 20
10.4 JIT와 warmup . . . 20
10.5 thread 상태 . . . 20
11. 스케줄러와 시간 귀속: 언제 느려졌는가 21
11.1 전체 p95는 주기적 spike를 숨긴다 . . . 21
11.2 tasks_scheduled 지표 읽기 . . . 21
11.3 시간 귀속의 네 단계 . . . 21
11.4 귀속(attribution)이 필요한 이유 . . . 22
12. 병목 진단 플레이북 23
12.1 출발점: 사용자 증상을 분류한다 . . . 23
12.2 CPU 병목 플레이북 . . . 23
12.3 메모리/OOM 플레이북 . . . 23
12.4 DB/pool 병목 플레이북 . . . 23
12.5 네트워크 병목 플레이북 . . . 23
12.6 병목을 못 찾았을 때 . . . 24
13. 반복 측정과 통계: CV를 노이즈로 읽으면 안 되는 이유 25
13.1 이번 프로젝트에서 확실히 확인된 사실 . . . 25
13.2 CV가 무엇을 계산하는가 . . . 25
13.3 왜 MDE가 틀어지는가 . . . 25

13.4 독립·동일분포(i.i.d.)라는 숨은 전제 . . . 26
13.5 현재 권장 실험 설계 . . . 26
13.6 ABBA를 숫자로 이해하기 . . . 26
13.7 비교 가능성 체크리스트 . . . 26
14. 현재 수집 카탈로그 165개 완전 해설 27
14.1 사전 사용법 . . . 27
14.2 CPU (cpu, 10개) . . . 27
14.3 Memory (memory, 7개) . . . 28
14.4 JVM Heap (heap, 9개) . . . 28
14.5 GC (gc, 10개) . . . 29
14.6 JVM 내부 (JIT / 스레드 / CPU) (jvm, 17개) . . . 30
14.7 MySQL (mysql, 28개) . . . 31
14.8 Redis (redis, 17개) . . . 33
14.9 Connection Pool / Threads (pool, 13개) . . . 34
14.10 Network (network, 4개) . . . 34
14.11 Disk (MySQL) (disk, 3개) . . . 35
14.12 호스트 (host, 11개) . . . 35
14.13 부하 발생기 (loadgen, 7개) . . . 36
14.14 k6 시계열 (remote-write) (k6ts, 13개) . . . 36
14.15 효율 (요청당 자원 비용) (efficiency, 8개) . . . 37
14.파생 포화도 8개 . . . 37
14.카탈로그를 실제로 읽는 세 가지 예 . . . 38
15. 앞으로 수집하면 좋은 지표를 배우는 법 39
15.1 가장 먼저 이해할 직접 실패 신호 . . . 39
15.2 원인을 갈라 주는 자원 지표 . . . 39
15.3 귀속 벡터: 어느 기능이 만들었나 . . . 39
15.4 k6에서 빠진 클라이언트 증거 . . . 40
15.5 SQL digest: 지표에서 코드로 가는 다리 . . . 40
15.6 조건부 심층 자료 . . . 40
15.7 기본 지표로 넣지 말아야 할 것 . . . 40
16. 실전 연습문제 24선 41
기초 문제 . . . 41
중급 문제 . . . 41
고급 문제 . . . 42
17. 연습문제 해설 44
부록 A. 증상별 1페이지 진단표 47
분석할 때 말로 읽는 순서 . . . 47

부록 B. 현재 임계값 빠른 참조 **48**

부록 C. 단위·공식·용어 **49**

단위와 공식 49

핵심 용어 49

부록 D. 4주 학습 루틴 **50**

최종 자기 점검 50

부록 E. 이 교재의 프로젝트 근거 **51**

끝 **51**

1. 병목 분석의 기본 언어

성능 병목은 '가장 높은 숫자'가 아니다. **수요가 공급 능력에 가까워져 대기가 생기고, 그 대기가 사용자 지연이나 처리량 한계로 전파되는 지점**이다. 그래서 사용량(utilization), 포화(saturation), 오류(errors), 지연(latency), 처리량(throughput)을 한 묶음으로 읽어야 한다.

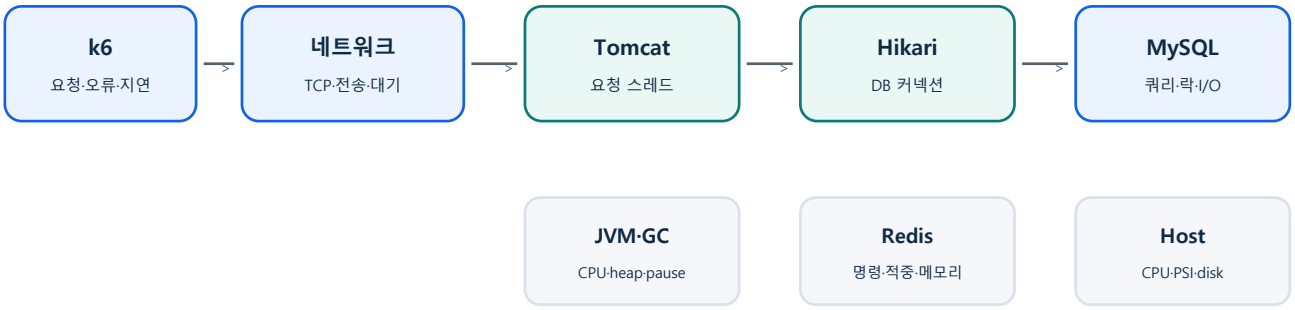


그림 1. 요청이 통과하는 주 경로와 옆에서 영향을 주는 자원. 병목은 한 상자에 머물지 않고 다음 상자의 대기로 전파된다.

1.1 다섯 개의 핵심 질문

축	쉬운 뜻	대표 예	함정
처리량	단위 시간에 끝낸 일	RPS, QPS, Redis ops/s	요청 종류가 다르면 같은 RPS도 비용이 다름
지연	일 하나가 끝날 때까지 걸린 시간	p50, p95, p99	평균만 보면 꼬리 지연을 숨김
사용량	자원을 얼마나 쓰는가	CPU cores, memory bytes	높음이 곧 병목은 아님
포화	한계에 얼마나 가까운가	busy/max, used/limit	분자·분모 범위가 다르면 가짜 100% 초과
오류	작업이 정상 완료되지 못한 정도	5xx, timeout, OOM, drop	누적 counter를 그대로 보면 과거 사건이 섞임

1.2 자원 부족과 비효율은 다르다

CPU 95%는 'CPU가 바쁘다'는 사실이다. 같은 RPS를 처리하면서 이전보다 CPU가 30% 더 들었다면 **비효율**일 수 있다. 반대로 CPU 95%인데 RPS도 선형으로 늘고 지연이 안정적이라면 자원을 잘 쓰는 중일 수 있다. 핵심은 절대 사용량과 작업당 비용을 분리하는 것이다.

> **요청당 CPU(ms) = CPU 사용 초 × 1000 / 완료 요청 수**
> 같은 종류의 요청, 같은 측정 구간이어야 비교할 수 있다.

이 프로젝트의 `efficiency.appCpuMsPerReq`, `jvmCpuMsPerReq`, `dbCpuMsPerReq`, `cacheCpuMsPerReq`, `stackCpuMsPerReq`는 바로 이 질문을 답한다. 처리량이 달라도 요청 하나를 처리하는 데 든 자원 비용을 비교할 수 있다.

1.3 관측 사실 → 가설 → 증명

단계	좋은 문장	나쁜 문장
사실	측정 구간에 Hikari pending이 0보다 컸다.	DB 풀이 병목이다.
가설	커넥션 부족 또는 보유 시간 증가가 요청 대기를 만들었을 수 있다.	풀 크기를 50으로 늘리면 해결된다.
증명	usage p95·SQL digest·pending 시간대가 겹치고 트랜잭션 축소 후 함께 감소했다.	pending이 있으니 원인은 DB다.

분석 문장의 품질 기준

'A가 높다'에서 멈추지 말고 'A가 높고 B도 같은 시간에 변했으므로 X를 의심한다. C가 그대로라면 Y는 약해진다. 다음에는 Z를 비교한다'까지 말한다.

2. Prometheus 지표를 읽는 법

2.1 Gauge, Counter, Histogram

형식	성질	읽는 법	프로젝트 예
Gauge	현재값이 오르내림	avg/max/p95로 구간 모양을 요약	메모리, active connections, threads
Counter	재시작 전까지 누적 증가	rate 또는 increase로 구간 변화량 계산	요청 수, CPU seconds, 오류 수
Histogram	값을 bucket별로 누적	histogram_quantile로 p95/p99 추정	HTTP, GC pause, Hikari acquire
Summary/Timer	클라이언트가 합계·개수·최댓값 제공	sum/count로 평균, max로 최악 확인	repository, lettuce command

Counter의 원시값 `56`은 '이번 실행에서 오류 56건'이 아니다. 앱이 시작된 뒤 지금까지 56건일 수 있다. 실행 구간의 오류는 `increase(counter[$RANGE])`로 읽는다. 프로세스 재시작과 counter reset도 Prometheus 함수가 처리하도록 해야 한다.

2.2 avg, max, p95가 서로 답하는 질문

요약	답하는 질문	잘 잡는 현상	놓치는 것
avg	구간 전체에서 보통 얼마나 썼나?	지속적 비용, 장기 경향	짧고 심한 spike
max	한 번이라도 어디까지 갔나?	위험한 단발 spike	지속성
p95	시간의 95%가 어느 수준 아래였나?	지속되는 높은 구간	상위 5%의 최악

> **예시**
>
> CPU avg 40%, p95 93%, max 100%라면 '평균 여유'가 아니라 반복적인 포화 구간이 있다는 뜻이다. avg 40%, p95 44%, max 100%라면 매우 짧은 단발 spike일 가능성이 크다.

2.3 p95는 무엇의 95분위인가

HTTP p95는 요청 표본의 95분위다. `cpu.cores.p95`는 시간축에서 긁은 CPU gauge 표본의 95분위다. 둘 다 p95지만 표본의 정체가 다르다. '요청 95%'와 '시간 95%'를 섞으면 안 된다.

2.4 분자와 분모의 범위를 맞춘다

포화도(%) = 같은 대상의 사용량 / 같은 대상의 한계 × 100

앱 컨테이너 메모리를 호스트 전체 메모리로 나누거나, 여러 컨테이너의 합을 하나의 limit로 나누면 의미가 깨진다.

현재 카탈로그는 컨테이너 재생성 직후 같은 이름의 이전 시계열이 잠시 겹칠 수 있어 `max(...)`로 한 컨테이너 값을 고른다. 물리적으로 100%를 지속 초과할 수 없는 포화도가 105%를 넘으면 자원 부족으로 해석하지 않고 측정 오류로 버린다.

2.5 결측은 0이 아니다

0은 '측정했고 발생하지 않음'이다. null 또는 UNMEASURED는 '측정할 근거가 없음'이다. exporter 장애, 라벨 불일치, scrape 공백이 생기면 좋은 값처럼 보이는 0으로 채우지 않는다. 측정 실패와 성능 실패는 서로 다른 축이다.

3. CPU: 바쁨, 포화, 제한, 효율

3.1 CPU core와 percent

`container_cpu_usage_seconds_total`의 초당 증가량은 사용한 코어 수다. 1초 동안 CPU 시간을 1초 썼으면 1 core, 2초 썼으면 2 cores다. 컨테이너 limit가 2 cores라면 1.4 cores는 70% 포화다.

> **CPU 포화도 = cpu.cores / cpu.limitCores × 100**
> 앱의 현재 limit는 2 cores. DB도 2 cores, Redis는 1 core로 제한돼 있다.

`process_cpu_usage`는 JVM 프로세스의 관점이고, `container_cpu_usage_seconds_total`은 컨테이너 전체 관점이다. 둘이 크게 다르면 JVM 밖의 사이드 프로세스, 계측 오버헤드, 라벨 범위 불일치를 확인한다.

3.2 throttling은 'CPU 사용률이 낮아도' 생길 수 있다

Linux CFS quota는 짧은 주기마다 컨테이너가 쓸 수 있는 CPU 시간을 배분한다. 한 주기에 허용량을 빨리 다 쓰면 남은 시간 동안 강제로 멈춘다. 전체 1분 평균 CPU는 70%여도 일부 주기에서 몰아 쓰면 throttling과 꼬리 지연이 생길 수 있다.

조합	해석	다음 확인
cores 높음 + throttle 높음 + p99 상승	CPU limit가 실제 요청을 막는 강한 증거	JFR hot method, limit 실험
cores 낮음 + throttle 높음	짧은 burst 또는 낮은 quota	1m 평균보다 짧은 시계열, scheduled burst
cores 높음 + throttle 0 + latency 안정	CPU를 효율적으로 쓰는 중일 수 있음	RPS가 선형인지 확인
cores 낮음 + latency 높음	CPU보다 I/O·lock·pool 대기 가능성	Hikari, MySQL lock, network

3.3 host CPU와 container CPU를 분리한다

컨테이너 CPU가 낮아도 호스트의 iowait가 높으면 디스크를 기다리느라 CPU가 놀 수 있다. steal이 높으면 가상화 호스트가 CPU를 다른 VM에 빼앗긴다. load per core가 높고 CPU가 낮다면 runnable task뿐 아니라 uninterruptible I/O wait가 섞였는지 확인한다.

3.4 CPU를 코드에 귀속한다

CPU 병목의 마지막 질문은 '누가 CPU를 썼나'다. 현재 스칼라 지표는 앱·JVM·DB·Redis의 총비용까지 알려준다. 메서드 수준 귀속은 JFR, SQL digest, endpoint별 latency, repository timer가 필요하다.

현상	가능한 원인	직접 증거
app CPU/req 증가	직렬화, 암호화, 정렬, 로깅, busy loop	JFR CPU flame graph
DB CPU/req 증가 + queries/req 동일	쿼리 한 건이 비싸짐	digest별 총시간·rows examined
DB CPU/req 증가 + queries/req 증가	N+1 또는 불필요 반복 쿼리	repository count, p6spy, digest count
cache CPU/req 증가	큰 자료구조·고비용 명령·키 공간 탐색	Lettuce command별 시간, Redis slowlog

> **현재 도구 정책**
>
> CPU saturation WARN > 80%, FAIL > 95%; 앱·MySQL throttled WARN > 1%, FAIL > 10%; load generator throttled WARN > 1%, FAIL > 5%. 이 값은 현재 운영 규칙이지 모든 시스템의 보편 법칙이 아니다.

4. 메모리: heap, RSS, working set, cache

4.1 같은 메모리를 다른 층에서 본다

지표	포함 범위	가장 잘 답하는 질문
heap.used	Java 객체가 쓰는 heap	객체 생존량과 GC 압력은 어떤가?
nonheap	Metaspace, Code Cache 등	클래스 로딩·동적 생성이 새는가?
RSS	프로세스가 실제 RAM에 올린 페이지	JVM 전체 물리 메모리 발자국은?
working set	최근 쓰인 cgroup 메모리, inactive file cache 제외	OOM limit에 실제로 얼마나 가까운가?
memory limit	cgroup 상한	강제 회수·OOM 경계는 어디인가?

`-Xmx=1g`라고 컨테이너 메모리를 1GB로 잡으면 안 된다. JVM은 heap 외에도 Metaspace, Code Cache, thread stack, direct buffer, native library를 쓴다. 현재 앱 컨테이너 limit 2560MB와 heap 1GB 사이의 여유가 이 영역을 수용한다.

4.2 누수와 캐시를 시간 모양으로 구분한다

모양	해석 후보	확인
heap.used 톱니 + GC 후 기준선 안정	정상 할당과 회수	live data 안정, GC overhead
GC 후 기준선이 계속 상승	live set 증가 또는 누수	liveDataMax, heap dump, 클래스/객체 상위
heap 안정 + RSS/working set 상승	direct buffer, thread stack, native, mmap	NMT/JFR, thread count, direct buffer
working set 상승 + OOM/failcnt 증가	cgroup limit 충돌	container OOM event, memory.events

4.3 MySQL과 Redis 메모리

MySQL의 2GB 컨테이너 안에서 InnoDB buffer pool은 1GB다. buffer pool hit ratio가 높아도 운영체제·연결 버퍼·정렬·임시 테이블 메모리가 필요하다. Redis는 컨테이너 512MB 안에서 `maxmemory 384mb`, `allkeys-lru`다. Redis memory saturation은 evictions와 함께 읽어야 한다.

> **오진 방지**
>
> 메모리 포화 100%에 가까워도 OOM이 없고 latency가 안정적이면 '메모리를 잘 활용'하는 중일 수 있다. 반대로 포화가 낮아도 failcnt·OOM event가 증가했다면 순간 peak나 잘못된 범위 집계가 있었는지 본다.

4.4 메모리 분석 순서

1. working set / limit로 cgroup 포화를 본다.
2. heap used / Xmx와 GC 후 live data로 Java 객체 압력을 본다.
3. RSS - heap 차이와 nonheap·threads로 native 영역 후보를 나눈다.
4. OOM event·failcnt·eviction 같은 직접 실패 신호를 확인한다.
5. 시간 추세가 특정 endpoint·scheduler·load phase와 겹치는지 확인한다.

5. 디스크와 I/O: 바쁜 디스크와 느린 디스크

5.1 bytes/sec와 latency는 다른 질문

read/write bytes per second는 '얼마나 옮겼나'를 말한다. io time은 디스크가 I/O를 처리하느라 바빴던 누적 시간이다. 둘만으로 요청 하나가 얼마나 기다렸는지 정확히 알 수 없다. 평균 응답시간(await)과 현재 queue depth가 있어야 '디스크가 느린가'를 직접 말할 수 있다.

평균 read await = rate(read_time_seconds_total) / rate(reads_completed_total)
분모가 충분한 구간에서만 의미가 있다. write도 같은 방식이다.

패턴	가능한 해석	함께 볼 것
bytes 높음, await 낮음	대역폭을 잘 처리 중	utilization, queue
bytes 낮음, await 높음	작은 랜덤 I/O, fsync, 장치 지연	fsync count, MySQL pending fsync
io time 높음, queue 증가	장치 포화	host iowait, DB latency
host iowait 높음, app CPU 낮음	CPU가 아니라 I/O 대기	PSI io, disk await

5.2 MySQL 쓰기 경로의 실제 비용

현재 perf MySQL은 `innodb_flush_log_at_trx_commit=1`과 `sync_binlog=1`을 사용한다. 내구성을 위해 커밋마다 redo log와 binlog 동기화를 요구하므로 write-heavy 시나리오에서 fsync 지연이 드러날 수 있다. `innodb_data_fsyncs`, `innodb_os_log_fsyncs`, pending fsyncs, log waits를 함께 보면 '쓰기량'과 '동기화 대기'를 분리할 수 있다.

5.3 임시 디스크 테이블

`mysql.tmpDiskTables` 증가는 메모리 임시 테이블이 디스크로 내려갔음을 뜻한다. 정렬·GROUP BY·큰 행·메모리 제한이 원인일 수 있다. 이 값만 보고 메모리를 늘리지 말고 digest에서 어떤 SQL이 임시 테이블과 많은 rows examined를 만들었는지 찾는다.

실험 설계

동일 write-heavy 부하에서 fsync 관련 지표와 latency가 함께 오르는지 먼저 본다. 내구성 설정을 바꾸는 실험은 원인 확인용으로만 하고, 운영 안전 요구를 무시한 채 성능 해결책으로 확정하지 않는다.

6. MySQL: 쿼리, 락, 버퍼, 커넥션

6.1 MySQL 병목을 네 갈래로 나눈다

갈래	대표 현상	직접 지표	귀속 증거
CPU	쿼리 계산·정렬·스캔이 비쌈	mysql.cpuCores, cpuSeconds	SQL digest, execution plan
I/O	버퍼 miss·fsync·임시 디스크	bufferPoolHit, disk await, fsync	digest rows examined, temp table
Lock	동시 쓰기가 같은 row를 기다림	row lock waits/time/deadlock	트랜잭션·잠금 SQL
Connection	앱이 커넥션을 못 얻거나 DB 상한 접근	Hikari pending, threadsConnected/max	usage time, transaction boundary

6.2 QPS/RPS는 N+1의 경보다

queriesPerReq = MySQL query 증가량 / HTTP request 증가량
요청 구성과 캐시 hit가 같을 때 비교한다.

`queriesPerReq`가 10이라고 해서 자동으로 N+1은 아니다. 한 비즈니스 요청이 원래 여러 SQL을 필요로 할 수 있다. 하지만 같은 endpoint·dataset에서 이 비율이 갑자기 늘면 코드가 추가 쿼리를 만들었을 가능성이 크다.

> **프로젝트 사례: 댓글 N+1**
>
> BTL-005는 댓글 경로의 N+1을 의심한다. `mysql.qps / k6.rps`는 간접 신호다. 결정적인 확인은 댓글 수 N을 바꿨을 때 repository 메서드 호출이나 SQL 수가 N에 비례하는지 보는 것이다.
> `PostReactionRepository.countByPostAndKindAndIsValidTrue` 같은 메서드별 count/sum은 범인을 좁힌다.

6.3 Buffer pool hit ratio를 과신하지 않는다

hit ratio는 논리 읽기 중 메모리에서 해결된 비율이다. 99.9%라도 전체 읽기가 매우 많으면 0.1% miss가 큰 I/O가 될 수 있다. 반대로 짧은 테스트에서 읽기 요청 수가 적으면 비율이 요동한다. hit ratio, rows read/sec, disk read bytes/await, endpoint latency를 함께 본다.

6.4 스캔과 정렬

지표	뜻	다음 질문
selectScan	첫 테이블을 full scan한 SELECT 수	어느 digest가 rows examined를 많이 만들었나?
selectFullJoin	조인에서 인덱스를 못 쓴 횟수	join key·복합 인덱스 순서가 맞나?
tmpDiskTables	디스크 임시 테이블 생성 수	정렬·GROUP BY·행 크기·메모리 제한은?
rowsReadPerSec	InnoDB가 읽은 row 속도	RPS 대비 비정상적으로 많은가?

6.5 락 지표의 시간 의미

`innodbRowLockWaits`는 현재 기다리는 수를 시간축으로 요약한 gauge다. `rowLockWaitCount`는 구간 중 새로 발생한 대기 횟수, `rowLockTimeMs`는 누적 대기시간 증가량이다. '대기가 자주 짧게'와 '적지만 길게'를 구분하려면 count와 time을 함께 본다.

평균 row lock wait = rowLockTimeMs / rowLockWaitCount
근사 평균이다. 꼬리를 숨기므로 가능하면 avg/max direct status와 digest/transaction 증거로 보완한다.

6.6 커넥션 수와 실행 쿼리 수

`threadsConnected`는 연결돼 있는 세션 수이고 `threadsRunning`은 실제로 쿼리를 수행 중인 세션 수다. connected가 높고 running이 낮으면 idle connection이 많을 수 있다. running이 CPU core 수보다 훨씬 높고 latency가 늘면 CPU·I/O·lock 대기로 queue가 생겼을 수 있다.

현재 도구 정책

buffer pool hit WARN < 99%, FAIL < 95%; aborted connects WARN > 0, FAIL > 10. 이 임계값은 표본 수와 시나리오를 함께 보고 해석한다.

7. 네트워크와 k6 시간 분해

7.1 클라이언트 duration을 조각낸다

k6 구간	무엇을 포함하나	높을 때 후보
blocked	연결 슬롯·소켓을 기다린 시간	클라이언트 연결 풀, VU 부족, 로컬 자원
connecting	TCP 연결 수립	네트워크, listen backlog, 포트 고갈
sending	요청 바이트 전송	큰 업로드, 클라이언트 NIC
waiting	요청 전송 후 첫 바이트까지	서버 queue + 앱 + DB/Redis 처리
receiving	첫 바이트 이후 응답 수신	큰 payload, 느린 네트워크, client 수신

`http_req_duration`은 보통 sending + waiting + receiving이다. blocked/connecting은 별도일 수 있다. `client p95 - server p95`를 정확한 네트워크 시간으로 부르면 안 된다. 두 p95는 서로 다른 요청 표본을 독립적으로 요약한 값이라 분위를 뺄 수 없고, client waiting에는 server queue도 포함된다.

7.2 bytes/sec와 packet error

network rx/tx bytes/sec는 트래픽 규모다. payload가 커져도 latency가 충분히 낮으면 병목이 아닐 수 있다. rx errors, tx dropped가 증가하면 드라이버·buffer·네트워크 경로 이상을 의심한다. counter는 실행 구간 증가량으로 읽는다.

7.3 TCP accept queue와 TIME_WAIT

지표 후보	뜻	사용 시점
ListenOverflows / ListenDrops	listen queue가 넘쳐 연결을 못 받음	connecting 상승·서버 p95 정상일 때
TCP_tw	TIME_WAIT 소켓 수	짧은 연결 폭증·포트 고갈 의심
conntrack entries/limit	커널 연결 추적 테이블 사용	다수 연결·NAT 환경
TCP retransmits	재전송된 세그먼트	packet loss·네트워크 품질 의심

7.4 load generator도 측정 대상이다

k6 컨테이너가 CPU throttling되거나 메모리가 부족하면 서버가 아니라 부하 발생기가 목표 RPS를 못 만든다. `k6ts.vus`, `loadgen.cores`, `loadgen.throttledPct`, `dropped_iterations`를 확인해야 '서버 한계'와 '발생기 한계'를 구분한다.

> **arrival-rate 핵심**
>
> dropped iterations가 늘면 설정한 arrival rate를 실행하지 못했다는 뜻이다. 이때 관측된 RPS가 평평하다고 서버 capacity라고 단정하면 안 된다. load generator CPU·VU·timeout을 함께 본다.

8. Tomcat과 Hikari: 두 개의 queue

8.1 요청은 두 번 줄을 설 수 있다

Tomcat은 HTTP 요청을 처리할 worker thread를 제공한다. DB가 필요한 요청은 다시 Hikari에서 DB connection을 빌린다. thread가 400개여도 connection이 10개라면 DB 구간에서는 최대 10개만 동시에 진행하고 나머지는 connection을 기다릴 수 있다.

위치	사용량	한계	대기 신호
Tomcat	tomcatBusy	tomcatMax	busy saturation, accept backlog
Hikari	hikariActive	hikariMax	hikariPending, acquire p95, timeout
MySQL	threadsRunning	CPU·I/O·max connections	row lock, query latency

8.2 현재 프로젝트의 구조적 불균형

> **관측된 설정**
>
> Tomcat `max-threads=400`, `accept-count=200`, `max-connections=10000`이 명시돼 있다. Hikari maximumPoolSize는 별도 설정이 없어 현재 진단 문서에서는 기본 10으로 본다. 이 40:1 차이는 곧바로 결함 확정이 아니라, 부하 시 pending·acquire·usage를 반드시 확인해야 하는 강한 구조적 가설이다.

Tomcat thread를 늘리면 CPU가 늘지 않아도 메모리와 context switch가 늘고, DB connection queue가 길어질 수 있다. Hikari pool을 늘리면 DB 동시성이 늘어 CPU·I/O·lock 경합이 더 심해질 수 있다. 어느 풀도 '클수록 좋다'가 아니다.

8.3 acquire와 usage가 답하는 질문

지표	질문	높을 때
acquire time	커넥션을 빌리기까지 얼마나 기다렸나?	pool 포화의 결과
usage time	빌린 뒤 반납까지 얼마나 오래 잡았나?	긴 transaction·외부 I/O·느린 SQL
pending	지금 기다리는 thread가 몇 개인가?	동시 queue의 직접 신호
timeout	기다리다 포기한 요청이 있었나?	직접 실패 신호

8.4 풀 확장보다 transaction diet를 먼저 본다

커넥션을 잡은 상태에서 S3 호출, Redis 호출, 큰 JSON 변환, 긴 반복문을 실행하면 DB는 놀아도 connection이 반환되지 않는다. KI-21은 S3 호출이 transaction 내부에 있어 connection 보유 시간이 길어질 수 있음을 지적한다. usage p95가 오르고 MySQL CPU·threadsRunning은 낮다면 이런 경계를 의심한다.

패턴	가설
pending ↑, acquire ↑, usage ↑, MySQL CPU 낮음	앱이 connection을 오래 보유
pending ↑, acquire ↑, usage ↑, MySQL CPU/lock ↑	DB 처리 또는 lock이 느려 보유시간 증가
Tomcat busy ↑, Hikari pending 0, CPU ↑	DB 이전/이후 앱 CPU 또는 다른 I/O
timeout ↑	사용자 실패가 발생한 직접 증거

현재 도구 정책

Hikari saturation WARN > 80%, FAIL > 95%; pending WARN > 0, FAIL > 5; acquire p95 WARN > 10ms, FAIL > 100ms; timeout은 1건부터 FAIL. Tomcat saturation WARN > 70%, FAIL > 90%.

9. Redis: hit ratio, 명령 비용, eviction

9.1 Redis는 빠르다는 전제를 버린다

Redis 서버가 빠른 것과 애플리케이션이 Redis 응답을 빨리 받는 것은 다르다. `redis.opsPerSec`는 서버 처리량이고 `lettuce_command_completion_seconds`는 앱이 명령을 보낸 뒤 완료될 때까지의 클라이언트 관점 시간이다. 네트워크·event loop·connection·서버 명령 실행이 함께 들어갈 수 있다.

9.2 hit ratio의 해석

hit ratio = keyspace_hits / (keyspace_hits + keyspace_misses) × 100
캐시가 아닌 용도의 명령까지 하나의 hit ratio로 설명할 수는 없다.

hit ratio가 내려가고 MySQL QPS·CPU가 함께 오르면 캐시 miss가 DB 부하로 전파됐다는 가설이 강해진다. hit ratio만 내려가고 DB 변화가 없다면 해당 cache가 핵심 경로가 아니거나 miss 시 DB를 읽지 않는 구조일 수 있다.

9.3 eviction과 expiration

지표	뜻	해석
evictedKeys	maxmemory 정책 때문에 강제로 제거	메모리 압력·working set 과대·TTL 부족
expiredKeys	TTL이 끝나 정상 제거	대개 정상 활동량; 증가 자체는 오류 아님
memoryUsed/max	Redis 자체 메모리 포화	eviction과 함께 봐야 위험 판단
blockedClients	blocking 명령 등으로 기다리는 client	명령 종류와 지속 여부 확인

9.4 프로젝트 사례: KEYS

`RedisViewCountStore`는 view count sync에서 `redisTemplate.keys(pattern)`을 호출하고, `ViewCountScheduler`는 60초 fixed delay로 실행된다. 현재 앱 메트릭에서는 재시작 이후 ViewCountScheduler 실행 58회와 Lettuce KEYS 58회가 관측돼 주기적 연결이 보인다.

> **무엇이 사실이고 무엇이 가설인가**
>
> 사실: KEYS가 실제 호출되고 스케줄러 횟수와 맞는다. 가설: keyspace가 커지면 O(N) KEYS가 Redis event loop를 오래 점유해 주기적 latency spike를 만들 수 있다. 증명: 같은 시각의 KEYS completion time·Redis CPU·endpoint p99를 겹쳐 보고, SCAN 기반 구현으로 바꾼 대조 실험이 필요하다.

Redis 서버 CPU가 낮아도 단일 thread에서 긴 명령 하나가 다른 명령을 막을 수 있다. 평균 CPU는 직렬화된 짧은 stall을 숨긴다. command별 completion max/p95와 slowlog가 중요한 이유다.

> **현재 도구 정책**
>
> Redis hit ratio WARN < 90%; eviction WARN > 0, FAIL > 1000. eviction 1건도 조사 신호지만 시나리오·정책에 따라 즉시 서비스 실패라는 뜻은 아니다.

10. JVM과 GC: 할당, 생존, 정지

10.1 GC를 세 질문으로 나눈다

질문	지표	뜻
얼마나 자주?	gc.count, allocation rate	객체를 얼마나 빠르게 만들고 회수하나
한 번에 얼마나 오래?	pause avg/p95/max	꼬리 지연을 만들 수 있는 STW 길이
전체 비용은?	total pause, overhead	실행 시간·CPU 중 GC가 차지한 몫
왜 점점 힘들어지나?	promotion, live data, heap post-GC	오래 사는 객체가 Old를 채우는가

10.2 pause max와 HTTP p99의 관계

GC pause max 300ms와 HTTP p99 300ms가 같다고 GC가 원인인 것은 아니다. 같은 시각에 pause가 발생했고, 여러 endpoint가 동시에 느려지고, CPU·DB·network가 설명하지 못할 때 원인 가설이 강해진다. 전체 실행에서 max 하나만 비교하면 시간 정렬이 빠져 있다.

10.3 Young와 Full을 분리한다

현재 GC 그룹은 Young와 Full/major를 합친 총계다. Young GC 수백 회 속에 Full GC 한 번이 묻힐 수 있다. `action="end of major GC"`를 따로 세어 `gc.fullCount`, `gc.fullPauseMaxMs`로 보존해야 'Full GC가 있었나'를 직접 답한다.

패턴	해석 후보
alloc rate ↑, young count ↑, live data 안정	단명 객체가 많음; 할당 최적화 후보
promotion ↑, live data ↑, full GC ↑	장수 객체 증가·캐시·누수 후보
pause ↑, heap 여유, CPU 포화	GC thread CPU 경쟁 또는 host 자원 압력
nonheap ↑, classesLoaded ↑	동적 클래스/클래스로더·Metaspace 후보

10.4 JIT와 warmup

JIT compilation time과 loaded classes가 초반에 크게 늘면 warmup이 끝나지 않은 상태일 수 있다. 반복 측정에서 첫 실행이 느리고 이후 안정되는 현상은 코드 최적화가 아니라 JIT·class loading·cache warmup의 결과일 수 있다. 비교 구간 전에 동일 warmup을 적용해야 한다.

10.5 thread 상태

BLOCKED는 Java monitor lock을 기다리는 thread다. WAITING에는 정상적인 pool idle thread도 많아 절대 개수만으로 병목을 말할 수 없다. 실행 간 증가, endpoint latency와의 동시성, thread dump stack을 함께 본다.

> **현재 도구 정책**
>
> GC pause max WARN > 200ms, FAIL > 500ms; GC overhead WARN > 5%, FAIL > 10%. 목표 latency가 100ms인 서비스라면 200ms 정책이 너무 느슨할 수 있다. 임계값은 SLO에서 역산한다.

11. 스케줄러와 시간 귀속: 언제 느려졌는가

11.1 전체 p95는 주기적 spike를 숨긴다

5분 테스트에서 60초마다 2초씩 느려져도 전체 요청의 5% 미만이면 p95가 거의 움직이지 않을 수 있다. 스케줄러 병목은 실행 전체 요약보다 time series·구간별 p99·scheduler 실행 시각을 겹쳐 봐야 한다.

프로젝트 작업	주기	경합 후보
ViewCountScheduler	완료 후 60초 fixed delay	Redis KEYS, DB sync, transaction
HotScoreScheduler	5분 fixed rate	Redis sorted set, 게시글 계산
TokenCleanupScheduler	설정된 정리 주기	DB delete/update, lock
SchoolMealScheduler	급식 갱신 주기	외부 API·DB write

프로젝트는 `@EnableScheduling`만 있고 전용 scheduler pool 설정이 없다. 기본 단일 scheduler에서 긴 작업 하나가 다음 작업 시작을 늦출 수 있다. 그러나 `executor_queued_tasks{name=messageBrokerTaskScheduler}=5`를 배치 backlog라고 읽으면 안 된다. 미래 heartbeat처럼 예약된 작업이 queue에 들어 있을 수 있고, 이름상 WebSocket broker scheduler다.

11.2 tasks_scheduled 지표 읽기

`tasks_scheduled_execution_seconds`는 @Scheduled 메서드별 count/sum/max를 제공한다. count는 실행 횟수, sum/count는 평균, max는 최악 실행시간이다. 예정 시각 대비 실제 시작 지연은 별도 지표가 필요할 수 있다.

11.3 시간 귀속의 네 단계

1. 사용자 지연 spike가 발생한 timestamp를 잡는다.
2. 같은 시간의 CPU·GC·pool·MySQL·Redis·disk 신호를 겹친다.
3. endpoint, repository, Redis command, scheduler 라벨로 비용 주체를 좁힌다.
4. 주기·입력 크기·작업 하나만 바꾼 실험으로 spike가 이동하거나 사라지는지 본다.

> **BTL-007의 핵심**
>
> '스케줄러가 있다'가 아니라, 스케줄러 실행 시각과 자원 spike·사용자 지연이 반복해서 정렬되는지가 핵심 증거다. 주기적 모양은 원인을 좁히는 강한 단서다.

11.4 귀속(attribution)이 필요한 이유

스칼라 `http p95=420ms`, `MySQL QPS=800`만으로는 어느 기능이 문제인지 모른다. 라벨 벡터를 제한적으로 보존하면 다음 질문을 답할 수 있다.

귀속 축	보존할 값	답하는 질문
HTTP endpoint	method·templated URI·status별 count/p95/max	어느 API가 느린가?
Repository	repository·method별 count/total/max	어느 데이터 접근 메서드가 비용을 만들었나?
Redis command	command별 count/total/max	KEYS, SET, LTRIM 중 무엇이 느린가?
Scheduler	class·method별 count/total/max	어느 배치가 spike와 겹치는가?

URI는 `/api/posts/123` 같은 실제 ID가 아니라 `/api/posts/{postId}` 템플릿으로 저장해야 cardinality가 통제된다. client name·remote address·exception class처럼 비교에 불필요한 라벨은 제거한다.

12. 병목 진단 플레이북

12.1 출발점: 사용자 증상을 분류한다

증상	첫 질문	우선 지표
p95/p99만 상승	꼬리에서만 대기가 생겼나?	max/p99, GC max, pool pending, time series
RPS가 더 안 늘어남	부하를 못 만든 건가, 서버가 포화된 건가?	dropped iterations, loadgen throttle, saturation
오류율 상승	어디서 실패했나?	status별 count, timeout, OOM, log increase
평균도 함께 상승	지속적 비용이 늘었나?	avg 자원, queries/req, CPU/req, DB time
일정 주기 spike	어떤 scheduler/GC/flush와 정렬되나?	scheduler, GC, Redis command, fsync

12.2 CPU 병목 플레이북

1. app·JVM·DB·Redis·loadgen 각각의 cores와 limit, throttling을 비교한다.
2. CPU/req로 작업당 비용이 변했는지 본다.
3. latency와 RPS가 CPU 변화와 같은 시간에 움직였는지 본다.
4. 앱이면 JFR, DB면 SQL digest, Redis면 command timing으로 귀속한다.
5. limit만 늘리는 실험과 코드 비용을 줄이는 실험을 분리한다.

12.3 메모리/OOM 플레이북

1. working set/limit, OOM event, failcnt로 직접 위험을 확인한다.
2. heap·nonheap·RSS를 나눠 JVM 내부/외부를 구분한다.
3. GC 후 live data 기준선과 시간 기울기를 본다.
4. Redis는 memory/max와 eviction, MySQL은 buffer pool과 container working set을 함께 본다.
5. heap dump/NMT/JFR는 재현 조건과 보관 크기를 통제해 수집한다.

12.4 DB/pool 병목 플레이북

1. Hikari pending·acquire·timeout으로 앱 쪽 queue 존재를 확인한다.
2. usage time으로 connection 보유가 길어진 원인을 나눈다.
3. MySQL CPU·threadsRunning·lock·disk await로 DB 내부 포화를 본다.
4. queries/req와 SQL digest로 N+1/비싼 한 쿼리를 구분한다.
5. 풀 크기 변경은 DB 포화를 악화시킬 수 있으므로 transaction 축소와 별도 실험한다.

12.5 네트워크 병목 플레이북

1. duration을 blocked/connecting/sending/waiting/receiving으로 분해한다.
2. server-side endpoint latency와 client waiting을 같은 시간창에서 비교한다.
3. ListenDrops, retransmits, rx/tx errors, TIME_WAIT를 확인한다.
4. payload size와 response bytes, client CPU/throttle을 확인한다.
5. 한 구간의 packet capture나 server timing으로 가설을 검증한다.

12.6 병목을 못 찾았을 때

상황	가능한 이유	다음 행동
모든 avg 정상	짧은 spike가 요약에 묻힘	time series·max·1초/5초 창
서버 자원 정상, client 느림	loadgen·network·payload	k6 breakdown, loadgen cgroup
스칼라만으로 귀속 불가	라벨을 접어 버림	endpoint/repository/command top-K
반복마다 방향성 변화	비정상 표본·환경 drift	13장의 blocked randomized design
지표 결측	관측 스택 문제	UNMEASURED로 분리, scrape/exporter 확인

종료 조건

원인을 찾았다는 말은 '지표가 높다'가 아니라, 원인 후보를 바꿨을 때 사용자 지표와 원인 지표가 예측한 방향으로 함께 변하고, 반대 가설은 설명력이 떨어졌다는 뜻이다.

13. 반복 측정과 통계: CV를 노이즈로 읽으면 안 되는 이유

13.1 이번 프로젝트에서 확실히 확인된 사실

세트	회차	Spearman	전체 변화	도구의 P95 CV
기준선 (08-16)	5	+0.90	+9.0%	3.12%
진단 (08-17)	8	-0.88	-6.3%	4.26%



그림 2. 실제 중간값을 재현한 그래프가 아니라, 관측된 단조 방향과 전체 변화율을 설명하기 위한 정규화 도식이다.

방향은 반대지만 두 세트 모두 반복 순서와 결과가 강하게 연결됐다. 즉 '같은 조건을 반복하면 값이 중심 주변에 무작위로 흩어진다'는 전제가 두 번 깨졌다. 기준선은 실행할수록 나빠졌고, 진단 세트는 실행할수록 좋아졌다.

13.2 CV가 무엇을 계산하는가

CV(%) = 표준편차 / 평균 × 100
값의 상대적 흩어짐 크기다. 흩어짐이 무작위인지, 시간 추세인지 구분하지 않는다.

예를 들어 100, 102, 104, 106, 109는 평균 주변에 작게 흩어진 것처럼 보여 CV가 낮을 수 있다. 하지만 실행 순서를 완벽히 섞어 보면 값이 계속 오른다. 이 변동은 '다음 측정도 무작위로 ±몇 %'라는 noise가 아니라 warmup 고갈, 열, 캐시 상태, 데이터 상태, background task 같은 시간 효과다.

13.3 왜 MDE가 틀어지는가

MDE(minimum detectable effect)는 현재 변동성을 바탕으로 '이 정도보다 큰 차이면 우연이 아닐 가능성이 높다'고 정하는 최소 효과 크기다. CV를 noise 추정치로 넣으면 반복값이 독립이고 동일한 분포에서 왔다는 가정이 숨어 있다.

상황	CV가 섞어 버리는 것	결과
실행할수록 9% 악화	시간 추세 + 작은 순간 noise	MDE가 noise를 과대/오해하고 순서 편향은 제거 못함
A를 먼저, B를 나중에 측정	코드 효과 + 시간 효과	B가 좋아 보이거나 나빠 보여도 코드 때문인지 모름
warmup이 매번 다름	초기화 상태 차이	평균과 CV가 실행 절차를 측정

핵심 결론

추세가 섞인 CV로 계산한 MDE는 '이 개선이 진짜인가'를 가르는 신뢰 가능한 선이 아니다. 먼저 추세의 원인을 제거하거나 실험 설계 안에서 시간 효과를 상쇄해야 한다.

13.4 독립·동일분포(i.i.d.)라는 숨은 전제

독립은 앞 실행이 뒤 실행에 영향을 주지 않는다는 뜻이다. 동일분포는 각 실행이 같은 평균과 변동 구조에서 나왔다는 뜻이다. 캐시가 쌓이거나 DB buffer pool이 데워지고, heap/Old가 축적되고, 호스트 온도나 background job이 달라지면 두 전제가 깨진다.

13.5 현재 권장 실험 설계

1. A와 B를 몰아서 하지 말고 ABBA, BAAB처럼 순서를 교차한다.
2. 시간 블록마다 A/B를 한 쌍으로 측정해 같은 시점의 환경을 공유시킨다.
3. 실행 전 warmup, dataset snapshot, container restart 여부, cooldown을 고정한다.
4. 각 실행의 순번과 시작 시각을 저장하고 Spearman/시간 추세를 먼저 검사한다.
5. raw 값과 시계열을 보존해 이상 실행을 사후 제외할 때 이유를 남긴다.
6. 추세가 남으면 단순 CV/MDE 대신 block별 A-B 차이, 회귀모형의 time term, 비모수 paired test를 검토한다.

13.6 ABBA를 숫자로 이해하기

환경이 매 실행마다 2ms씩 느려진다고 가정하자. A를 4번 먼저 측정하고 B를 4번 나중에 측정하면 코드가 같아도 B가 약 8ms 느려 보인다. ABBA 순서는 A와 B가 앞·뒤 시간을 나눠 가지므로 선형 시간 drift의 영향을 줄인다. 무작위화는 모든 문제를 없애지 않지만, 처리 효과와 실행 순서가 완전히 겹치는 일을 막는다.

13.7 비교 가능성 체크리스트

범주	같아야 할 조건
부하	시나리오, arrival rate/VU 선언값, duration, 데이터 분포
애플리케이션	commit/build, JVM 옵션, profile, warmup
데이터	dataset snapshot/hash, cache 초기 상태, DB 통계
인프라	CPU/memory limit, host, container image, exporter
측정	수집 창, scrape interval, query definition, timeout/censoring

추가 관측

E-46 조사에서는 같은 조건으로 분류된 세션 A→C 사이에 p95가 약 45% 증가하고 RPS가 약 25% 감소했다. seriesHash와 dataset이 같아도 host pressure·JIT·cache·scheduler·열 상태처럼 아직 지문에 없는 조건이 달랐을 수 있다.

14. 현재 수집 카탈로그 165개 완전 해설

> **2026-08-18 현재 워킹트리 기준**
>
> 카탈로그 원시·요약 키 157개와 파생 포화도 8개, 합계 165개다. 과거 조사 문서의 '110개'는 이후 추가된 JVM·MySQL/Redis CPU·k6 time series·효율 지표를 반영하지 않은 이전 숫자다.

아래 표는 `performance/tools/lib/metrics-catalog.js`를 PDF 생성 시 직접 읽어 만든다. 따라서 키·표시명·단위·카탈로그 설명은 현재 파일과 일치한다. 각 키마다 '읽는 법'과 '함께 볼 증거'를 덧붙였다.

14.1 사전 사용법

키 접미사	표본	해석
.avg	시간축 gauge 평균	실행 전체의 지속적 수준
.max	시간축 gauge 최댓값	한 번이라도 발생한 최악 spike
.p95	시간축 gauge 95분위	시간의 95%가 이 값 이하
increase 기반 count/seconds	실행 구간 누적 증가량	구간 중 발생한 총 사건·비용
rate 기반 per_sec	초당 변화율	처리 속도·대역폭

14.2 CPU (cpu, 10개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
cpu.process.avg	percent	JVM이 사용한 CPU 비율. 호스트 전체 코어 기준.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
cpu.process.max	percent	JVM이 사용한 CPU 비율. 호스트 전체 코어 기준.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
cpu.process.p95	percent	JVM이 사용한 CPU 비율. 호스트 전체 코어 기준.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
cpu.cores.avg	cores	컨테이너가 소비한 CPU 코어 수. cgroup 한계와 직접 비교한다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
cpu.cores.max	cores	컨테이너가 소비한 CPU 코어 수. cgroup 한계와 직접 비교한다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
cpu.cores.p95	cores	컨테이너가 소비한 CPU 코어 수. cgroup 한계와 직접 비교한다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
cpu.limitCores	cores	cgroup에 설정된 CPU 상한. 포화도 계산의 분모.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
cpu.throttledPct	percent	cgroup이 CPU를 강제로 뺏은 주기 비율. >0 이면 CPU 한계가 지연에 직접 영향을 준다.	cgroup 주기 중 CPU 강제 정지가 발생한 비율. / cores/limit·latency·CPU/req
cpu.seconds	sec	측정 구간에 앱 컨테이너가 태운 CPU 초. 요청 수로 나누면 요청당 CPU 비용.	실행 구간의 총 CPU 비용. / 요청 수로 나눈 CPU/req
cpu.throttledSeconds	seconds	테스트 구간 동안 강제 정지된 총 시간.	실행 중 강제 정지된 누적 시간. / throttledPct·실행 duration

14.3 Memory (memory, 7개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
memory.workingSet.avg	bytes	OOM 판정 기준이 되는 실사용 메모리(page cache 제외).	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
memory.workingSet.max	bytes	OOM 판정 기준이 되는 실사용 메모리(page cache 제외).	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
memory.workingSet.p95	bytes	OOM 판정 기준이 되는 실사용 메모리(page cache 제외).	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
memory.limitBytes	bytes	cgroup 메모리 상한.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
memory.rss.avg	bytes	프로세스가 물리 메모리에 올린 양.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
memory.rss.max	bytes	프로세스가 물리 메모리에 올린 양.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
memory.rss.p95	bytes	프로세스가 물리 메모리에 올린 양.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수

14.4 JVM Heap (heap, 9개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
heap.used.avg	bytes	힙 영역 합계 사용량.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
heap.used.max	bytes	힙 영역 합계 사용량.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
heap.used.p95	bytes	힙 영역 합계 사용량.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
heap.maxBytes	bytes	-Xmx 로 설정된 힙 상한.	메모리 발자국과 상한 접근 정도. / OOM/failcnt·GC·시간 기울기
heap.committedMax	bytes	OS로부터 실제로 확보한 힙 크기.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
heap.liveDataMax	bytes	Full GC 직후 남은 힙. 계속 우상향하면 누수 의심.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
heap.nonHeapUsed.avg	bytes	Metaspace/CodeCache 등.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
heap.nonHeapUsed.max	bytes	Metaspace/CodeCache 등.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
heap.nonHeapUsed.p95	bytes	Metaspace/CodeCache 등.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수

14.5 GC (gc, 10개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
gc.pauseAvgMs	ms	한 번의 GC가 애플리케이션을 멈춘 평균 시간.	Stop-The-World가 사용자 지연에 줄 수 있는 영향. / HTTP p99 시간 정렬·GC action
gc.pauseMaxMs	ms	최악의 단일 STW 시간. p99 지연의 주범인지 판단하는 근거.	Stop-The-World가 사용자 지연에 줄 수 있는 영향. / HTTP p99 시간 정렬·GC action
gc.pauseP95Ms	ms	GC pause 분포의 95분위.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
gc.count	count	테스트 구간 총 GC 발생 횟수.	할당·회수·생존 객체가 만드는 GC 압력. / heap·live data·HTTP tail latency
gc.totalPauseMs	ms	구간 내 STW 누적. duration 대비 비율이 GC 오버헤드다.	Stop-The-World가 사용자 지연에 줄 수 있는 영향. / HTTP p99 시간 정렬·GC action
gc.overheadPct.avg	percent	CPU 시간 중 GC가 차지한 비율. 지속 5% 초과면 튜닝 대상.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
gc.overheadPct.max	percent	CPU 시간 중 GC가 차지한 비율. 지속 5% 초과면 튜닝 대상.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
gc.overheadPct.p95	percent	CPU 시간 중 GC가 차지한 비율. 지속 5% 초과면 튜닝 대상.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
gc.allocRateBytesSec	bytes_per_sec	초당 힙 할당량. GC 빈도의 선행 지표.	할당·회수·생존 객체가 만드는 GC 압력. / heap·live data·HTTP tail latency
gc.promotedBytesSec	bytes_per_sec	Young에서 Old로 승격된 초당 바이트. 높으면 Full GC가 다가온다.	할당·회수·생존 객체가 만드는 GC 압력. / heap·live data·HTTP tail latency

14.6 JVM 내부 (JIT / 스레드 / CPU) (jvm, 17개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
jvm.jitCompileMs	ms	JIT 컴파일러가 쓴 CPU 시간. 워밍업이 덜 끝났으면 크고, 정상 상태면 작다.	JVM 내부 warmup·CPU·thread 활동. / container CPU·JFR
jvm.classesLoaded.avg	count	현재 로드된 클래스 수. 워밍업 진행도의 대리 지표.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
jvm.classesLoaded.max	count	현재 로드된 클래스 수. 워밍업 진행도의 대리 지표.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
jvm.classesLoaded.p95	count	현재 로드된 클래스 수. 워밍업 진행도의 대리 지표.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
jvm.cpuSeconds	sec	JVM 프로세스가 실제로 쓴 CPU 초. 요청 수로 나누면 요청당 CPU 비용.	실행 구간의 총 CPU 비용. / 요청 수로 나눈 CPU/req
jvm.cpuUsagePct.avg	percent	JVM 이 쓰는 CPU 비율(호스트 코어 기준).	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
jvm.cpuUsagePct.max	percent	JVM 이 쓰는 CPU 비율(호스트 코어 기준).	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
jvm.cpuUsagePct.p95	percent	JVM 이 쓰는 CPU 비율(호스트 코어 기준).	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
jvm.threadsLive.avg	count	전체 스레드 수. 누수가 있으면 실행마다 증가한다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
jvm.threadsLive.max	count	전체 스레드 수. 누수가 있으면 실행마다 증가한다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
jvm.threadsLive.p95	count	전체 스레드 수. 누수가 있으면 실행마다 증가한다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
jvm.threadsBlocked.avg	count	synchronized 락을 기다리는 스레드. >0 이 지속되면 앱 레벨 경합이다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
jvm.threadsBlocked.max	count	synchronized 락을 기다리는 스레드. >0 이 지속되면 앱 레벨 경합이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
jvm.threadsBlocked.p95	count	synchronized 락을 기다리는 스레드. >0 이 지속되면 앱 레벨 경합이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
jvm.threadsWaiting.avg	count	대기 중인 스레드. 커넥션 풀·큐 대기가 여기 잡힌다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
jvm.threadsWaiting.max	count	대기 중인 스레드. 커넥션 풀·큐 대기가 여기 잡힌다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
jvm.threadsWaiting.p95	count	대기 중인 스레드. 커넥션 풀·큐 대기가 여기 잡힌다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수

14.7 MySQL (mysql, 28개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
mysql.threadsRunning.avg	count	실제로 쿼리를 실행 중인 스레드. DB 동시성의 직접 지표.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
mysql.threadsRunning.max	count	실제로 쿼리를 실행 중인 스레드. DB 동시성의 직접 지표.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
mysql.threadsRunning.p95	count	실제로 쿼리를 실행 중인 스레드. DB 동시성의 직접 지표.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
mysql.threadsConnected.avg	count	연결된 커넥션 수.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
mysql.threadsConnected.max	count	연결된 커넥션 수.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
mysql.threadsConnected.p95	count	연결된 커넥션 수.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
mysql.maxConnections	count	서버 커넥션 상한. 포화도 계산 분모.	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series
mysql.slowQueries	count	long_query_time(0.1s) 초과 쿼리 수. 인덱스 회귀의 1차 신호.	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series
mysql.qps	per_sec	초당 쿼리 수. RPS 대비 비율이 곧 요청당 쿼리 수 = N+1 탐지기.	DB가 처리한 쿼리 속도. 요청 비용과 함께 읽는다. / k6 RPS·queriesPerReq
mysql.qpsMax	per_sec	순간 최대 쿼리 처리율.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
mysql.bufferPoolHitPct	percent	InnoDB 버퍼풀 적중률. 낮으면 디스크에서 읽고 있다는 뜻.	논리 읽기 중 buffer pool에서 해결된 비율. / reads·disk await·표본 수
mysql.rowsReadPerSec	per_sec	초당 읽은 행 수. 쿼리 수 대비 급증하면 풀스캔 의심.	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series
mysql.abortedConnects	count	연결 실패 횟수. 커넥션 고갈의 직접 증거.	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series
mysql.innodbRowLockWaits.avg	count	현재 행 잠금 대기 수. 핫 로우 경합 탐지.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
mysql.innodbRowLockWaits.max	count	현재 행 잠금 대기 수. 핫 로우 경합 탐지.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
mysql.innodbRowLockWaits.p95	count	현재 행 잠금 대기 수. 핫 로우 경합 탐지.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
mysql.selectScan	count	첫 테이블을 전부 훑은 SELECT 수. 급증하면 인덱스가 안 먹고 있다.	인덱스를 충분히 쓰지 못한 작업의 간접 신호. / SQL digest·rows examined·EXPLAIN
mysql.selectFullJoin	count	조인 키에 인덱스가 없어 상대 테이블을 전부 훑은 횟수. 0이어야 정상.	인덱스를 충분히 쓰지 못한 작업의 간접 신호. / SQL digest·rows examined·EXPLAIN
mysql.tmpDiskTables	count	메모리에 못 담아 디스크로 내려간 임시 테이블. ORDER BY/GROUP BY 비용의 신호.	임시 작업이 메모리를 벗어나 디스크를 사용한 횟수. / digest·disk await·sort/group by
mysql.rowLockTimeMs	ms	행 잠금을 기다린 누적 시간(ms). 응답시간에 그대로 더해진다. (T-29)	동시 트랜잭션이 같은 자원을 기다린 흔적. / 대기 count/time·SQL·transaction
mysql.rowLockWaitCount	count	행 잠금 대기가 발생한 횟수. 대기 총시간과 나누면 1회 평균 대기. (T-29)	동시 트랜잭션이 같은 자원을 기다린 흔적. / 대기 count/time·SQL·transaction
mysql.rollbacks	count	롤백된 트랜잭션 수. 데드락·예외가 조용히 늘고 있는지 본다. (T-29)	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
mysql.cpuCores.avg	cores	MySQL 컨테이너가 소비한 CPU 코어 수. 앱 CPU 와 나란히 놓고 본다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
mysql.cpuCores.max	cores	MySQL 컨테이너가 소비한 CPU 코어 수. 앱 CPU 와 나란히 놓고 본다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
mysql.cpuCores.p95	cores	MySQL 컨테이너가 소비한 CPU 코어 수. 앱 CPU 와 나란히 놓고 본다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
mysql.cpuSeconds	sec	측정 구간에 MySQL 이 태운 CPU 초. 쿼리 수로 나누면 쿼리당 CPU 비용.	실행 구간의 총 CPU 비용. / 요청 수로 나눈 CPU/req
mysql.cpuLimitCores	cores	MySQL 에 걸린 cgroup CPU 상한. 여유가 있는데 느리면 CPU 부족이 아니다.	정의된 단위와 수집 구간 안에서 변화 방향을 본다. / 같은 자원의 상관 지표·time series
mysql.throttledPct	percent	MySQL 이 cgroup 에 강제 정지된 주기 비율. >0 이면 DB 지연의 원인이 CPU 상한이다.	cgroup 주기 중 CPU 강제 정지가 발생한 비율. / cores/limit·latency·CPU/req

14.8 Redis (redis, 17개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
redis.opsPerSec	per_sec	초당 처리 명령 수.	Redis 서버의 처리량·메모리·실패 신호. / Lettuce command별 시간·앱 latency
redis.cpuCores.avg	cores	Redis 컨테이너가 소비한 CPU 코어 수.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
redis.cpuCores.max	cores	Redis 컨테이너가 소비한 CPU 코어 수.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
redis.cpuCores.p95	cores	Redis 컨테이너가 소비한 CPU 코어 수.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
redis.cpuSeconds	sec	측정 구간에 Redis 가 태운 CPU 초.	실행 구간의 총 CPU 비용. / 요청 수로 나눈 CPU/req
redis.opsPerSecMax	per_sec	순간 최대 처리율.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
redis.connectedClients.avg	count	연결된 클라이언트 수. Lettuce 풀 설정과 비교한다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
redis.connectedClients.max	count	연결된 클라이언트 수. Lettuce 풀 설정과 비교한다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
redis.connectedClients.p95	count	연결된 클라이언트 수. Lettuce 풀 설정과 비교한다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
redis.hitRatioPct	percent	캐시 적중률. 캐시 기여도 실험(EXP-004)의 핵심 지표.	Redis 서버의 처리량·메모리·실패 신호. / Lettuce command별 시간·앱 latency
redis.evictedKeys	count	메모리 부족으로 쫓겨난 키 수. >0 이면 캐시 효과가 무너지고 있다.	Redis 서버의 처리량·메모리·실패 신호. / Lettuce command별 시간·앱 latency
redis.expiredKeys	count	TTL 만료로 삭제된 키 수.	Redis 서버의 처리량·메모리·실패 신호. / Lettuce command별 시간·앱 latency
redis.memoryUsed.avg	bytes	Redis가 사용 중인 메모리.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
redis.memoryUsed.max	bytes	Redis가 사용 중인 메모리.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
redis.memoryUsed.p95	bytes	Redis가 사용 중인 메모리.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
redis.memoryMaxBytes	bytes	Redis 메모리 상한 (0이면 무제한).	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
redis.blockedClients	count	블로킹 명령 대기 클라이언트.	동시 트랜잭션이 같은 자원을 기다린 흔적. / 대기 count/time·SQL·transaction

14.9 Connection Pool / Threads (pool, 13개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
pool.hikariActive.avg	count	사용 중인 DB 커넥션.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
pool.hikariActive.max	count	사용 중인 DB 커넥션.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
pool.hikariActive.p95	count	사용 중인 DB 커넥션.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
pool.hikariPending.avg	count	커넥션을 기다리는 스레드. >0 이 지속되면 풀이 병목이다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
pool.hikariPending.max	count	커넥션을 기다리는 스레드. >0 이 지속되면 풀이 병목이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
pool.hikariPending.p95	count	커넥션을 기다리는 스레드. >0 이 지속되면 풀이 병목이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
pool.hikariMax	count	풀 최대 크기. 포화도 분모.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인
pool.hikariAcquireP95Ms	ms	풀에서 커넥션을 받기까지 걸린 p95. 응답시간에 그대로 더해진다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
pool.hikariTimeouts	count	커넥션을 못 받고 실패한 횟수. 0이어야 정상.	DB connection pool의 사용·대기·실패 상태. / usage/acquire/pending·MySQL
pool.tomcatBusy.avg	count	요청 처리 중인 워커 스레드.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
pool.tomcatBusy.max	count	요청 처리 중인 워커 스레드.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
pool.tomcatBusy.p95	count	요청 처리 중인 워커 스레드.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
pool.tomcatMax	count	워커 스레드 상한. 여기 닿으면 요청이 큐에서 대기한다.	사용량을 해석할 분모 또는 상한. / 같은 대상 사용량과 범위 일치 확인

14.10 Network (network, 4개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
network.rxBytesSec	bytes_per_sec	앱 컨테이너 초당 수신 바이트.	컨테이너 NIC의 트래픽 또는 오류 변화. / k6 breakdown·host TCP
network.txBytesSec	bytes_per_sec	앱 컨테이너 초당 송신 바이트.	컨테이너 NIC의 트래픽 또는 오류 변화. / k6 breakdown·host TCP
network.rxErrors	count	네트워크 수신 오류. 0이어야 정상.	컨테이너 NIC의 트래픽 또는 오류 변화. / k6 breakdown·host TCP
network.txDropped	count	드롭된 송신 패킷.	컨테이너 NIC의 트래픽 또는 오류 변화. / k6 breakdown·host TCP

14.11 Disk (MySQL) (disk, 3개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
disk.readBytesSec	bytes_per_sec	DB 컨테이너 초당 디스크 읽기. 버퍼풀 미스와 연동해서 본다.	디스크 전송량 또는 busy 시간. / await·queue·iowait
disk.writeBytesSec	bytes_per_sec	DB 컨테이너 초당 디스크 쓰기 (redo/binlog 포함).	디스크 전송량 또는 busy 시간. / await·queue·iowait
disk.ioTimeSec	seconds	디스크가 바빴던 누적 시간. duration 대비 비율이 디스크 이용률.	디스크 전송량 또는 busy 시간. / await·queue·iowait

14.12 호스트 (host, 11개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
host.cores	cores	VM 에 할당된 논리 코어 수. 아래 사용률의 분모.	컨테이너 바깥 호스트 경쟁과 여유. / container 지표와 같은 시간창
host.cpuPct.avg	percent	VM 전체 CPU 사용률. 개별 컨테이너가 한가해도 이 값이 높으면 서로 밀어낸 것이다.	구간의 평상시 수준. 지속 비율을 본다. / 같은 계열 p95·max와 비교
host.cpuPct.max	percent	VM 전체 CPU 사용률. 개별 컨테이너가 한가해도 이 값이 높으면 서로 밀어낸 것이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
host.cpuPct.p95	percent	VM 전체 CPU 사용률. 개별 컨테이너가 한가해도 이 값이 높으면 서로 밀어낸 것이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
host.loadPerCore.avg	ratio	load1 을 코어 수로 나눈 값. 1 을 넘으면 실행 대기가 코어보다 많다는 뜻이다.	구간의 평상시 수준. 지속 비율을 본다. / 같은 계열 p95·max와 비교
host.loadPerCore.max	ratio	load1 을 코어 수로 나눈 값. 1 을 넘으면 실행 대기가 코어보다 많다는 뜻이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
host.loadPerCore.p95	ratio	load1 을 코어 수로 나눈 값. 1 을 넘으면 실행 대기가 코어보다 많다는 뜻이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
host.iowaitPct	percent	CPU 가 디스크를 기다린 비율. 높으면 병목이 CPU 가 아니라 I/O 다.	컨테이너 바깥 호스트 경쟁과 여유. / container 지표와 같은 시간창
host.stealPct	percent	하이퍼바이저가 CPU 를 가져간 비율. VM 밖(Windows) 부하의 간접 증거다.	컨테이너 바깥 호스트 경쟁과 여유. / container 지표와 같은 시간창
host.memAvailableBytes	bytes	구간 중 최저 가용 메모리. 0 에 가까우면 페이지 캐시가 밀려 디스크 읽기가 는다.	컨테이너 바깥 호스트 경쟁과 여유. / container 지표와 같은 시간창
host.contextSwitchesSec	per_sec	초당 컨텍스트 스위치. 급증은 과도한 스레드 경합 신호다.	컨테이너 바깥 호스트 경쟁과 여유. / container 지표와 같은 시간창

14.13 부하 발생기 (loadgen, 7개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
loadgen.cores.avg	cores	k6 컨테이너가 소비한 코어 수. 측정 대상과 같은 머신이면 이만큼 뺏은 것이다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
loadgen.cores.max	cores	k6 컨테이너가 소비한 코어 수. 측정 대상과 같은 머신이면 이만큼 뺏은 것이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
loadgen.cores.p95	cores	k6 컨테이너가 소비한 코어 수. 측정 대상과 같은 머신이면 이만큼 뺏은 것이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
loadgen.throttledPct	percent	**0 이 아니면 그 실행의 지연은 서버가 아니라 부하 발생기가 만든 것일 수 있다.** 서버 포화와 발생기 포화를 가르는 값.	cgroup 주기 중 CPU 강제 정지가 발생한 비율. / cores/limit·latency·CPU/req
loadgen.memBytes.avg	bytes	k6 가 쓴 메모리. VU 가 많으면 여기가 먼저 터진다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
loadgen.memBytes.max	bytes	k6 가 쓴 메모리. VU 가 많으면 여기가 먼저 터진다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
loadgen.memBytes.p95	bytes	k6 가 쓴 메모리. VU 가 많으면 여기가 먼저 터진다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수

14.14 k6 시계열 (remote-write) (k6ts, 13개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
k6ts.p95	ms	측정 구간 전체의 응답시간 P95. k6 요약본의 P95 와 같은 값이어야 한다 — 어긋나면 측정 창 계산을 의심한다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
k6ts.p99	ms	응답시간 P99. 꼬리가 얼마나 두꺼운지를 본다.	사용자가 본 처리량·지연·오류의 직접 결과. / 서버 endpoint·자원 time series
k6ts.p50	ms	응답시간 중앙값. P95 와의 벌어짐이 곧 꼬리 비대칭의 크기다.	사용자가 본 처리량·지연·오류의 직접 결과. / 서버 endpoint·자원 time series
k6ts.p95Read	ms	읽기 요청만의 P95. SLO 기준 300ms.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
k6ts.p95Write	ms	쓰기 요청만의 P95. SLO 기준 500ms.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
k6ts.waitingP95	ms	첫 바이트까지 기다린 시간의 P95(TTFB). P95 전체와 거의 같으면 지연은 서버 안에서 났다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
k6ts.blockedP95	ms	요청을 보내기 전 커넥션을 기다린 시간. 커지면 부하 발생기 쪽 병목이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수
k6ts.rps	per_sec	측정 구간의 초당 요청 수. 닫힌 부하 루프라 지연이 오르면 이 값이 내려간다.	사용자가 본 처리량·지연·오류의 직접 결과. / 서버 endpoint·자원 time series
k6ts.reqs	count	측정 구간 요청 총수. k6 요약본의 measure 구간 httpReqs 와 대조하는 용도.	사용자가 본 처리량·지연·오류의 직접 결과. / 서버 endpoint·자원 time series
k6ts.errorPct	percent	측정 구간의 실패 응답 비율. 구간을 잘라도 정확하게 다시 계산된다.	사용자가 본 처리량·지연·오류의 직접 결과. / 서버 endpoint·자원 time series
k6ts.vus.avg	count	그 순간 붙어 있던 가상 사용자 수. 램프업 곡선과 지연 상승을 겹쳐 보는 축이다.	구간의 평상시 수준. 지속 비용을 본다. / 같은 계열 p95·max와 비교
k6ts.vus.max	count	그 순간 붙어 있던 가상 사용자 수. 램프업 곡선과 지연 상승을 겹쳐 보는 축이다.	한 번이라도 도달한 최악. 단발 spike에 민감하다. / p95와 time series로 지속성 확인
k6ts.vus.p95	count	그 순간 붙어 있던 가상 사용자 수. 램프업 곡선과 지연 상승을 겹쳐 보는 축이다.	상위 5%를 제외한 높은 수준. 꼬리의 지속성을 본다. / avg·max·표본 수

14.15 효율 (요청당 자원 비용) (efficiency, 8개)

키	단위	정의·물리적 의미	읽는 법 / 함께 볼 증거
efficiency.appCpuMsPerReq	ms	요청 1건에 앱 컨테이너가 태운 CPU(ms). **CPU 포화 상태에서 성능 회귀를 직접 보는 값.** 오르면 같은 일을 더 비싸게 하고 있다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.jvmCpuMsPerReq	ms	JVM 프로세스만의 요청당 CPU(ms). 앱 컨테이너 값과 크게 다르면 컨테이너 안에 다른 소비자가 있다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.dbCpuMsPerReq	ms	요청 1건에 MySQL 이 태운 CPU(ms). 앱 값과 **같은 비율로** 움직이면 원인은 둘 다의 아래 층이다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.cacheCpuMsPerReq	ms	요청 1건에 Redis 가 태운 CPU(ms).	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.stackCpuMsPerReq	ms	앱+MySQL+Redis 를 합친 요청당 CPU. 병목이 옮겨 다녀도 이 합계는 총비용을 그대로 보여준다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.dbCpuUsPerQuery	us	쿼리 1건에 든 CPU(마이크로초). 요청당 쿼리 수는 그대로인데 이 값만 오르면 DB 자체가 비효율해진 것이다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.queriesPerReq	count	요청 1건이 만든 쿼리 수. 급증하면 N+1 이다. 위 두 지표를 해석할 때의 기준선.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline
efficiency.ctxSwitchPerReq	count	요청 1건당 VM 전체 컨텍스트 스위치. CPU 는 그대로인데 이게 오르면 스케줄링 비용이 늘어난 것이다.	요청 하나당 소비한 자원 비용. / 절대 사용량·요청 구성·baseline

14.파생 포화도 8개

파생값은 새 센서가 아니라 카탈로그 값의 비율이다. 분자·분모 중 하나가 결측이거나 분모가 0이면 null이다. 105%를 넘으면 컨테이너 재생성 뒤 중복 시계열 등 측정 오류로 간주해 버린다.

키	계산	답하는 질문	함께 볼 것
saturation.cpuPct	앱 CPU max / 앱 CPU limit	앱 CPU의 최악 포화도	throttle·latency
saturation.cpuAvgPct	앱 CPU avg / 앱 CPU limit	앱 CPU의 지속 포화도	cpuPct와 비교
saturation.memoryPct	working set max / memory limit	앱 cgroup 메모리 포화	OOM·failcnt
saturation.heapPct	heap used max / Xmx	Java heap 포화	GC·live data
saturation.hikariPct	Hikari active max / pool max	DB pool 포화	pending·acquire·usage
saturation.tomcatPct	Tomcat busy max / thread max	HTTP worker 포화	Hikari·CPU
saturation.mysqlConnPct	MySQL connected max / max_connections	DB 연결 상한 포화	Hikari·running
saturation.redisMemPct	Redis used max / maxmemory	Redis 메모리 포화	eviction·hit ratio

14.카탈로그를 실제로 읽는 세 가지 예

예 1: `saturation.cpuPct=92`, `cpu.throttledPct=0`, `appCpuMsPerReq` 안정

앱 CPU는 limit에 가깝지만 강제 정지는 없고 작업당 비용도 변하지 않았다. RPS가 증가한 결과일 수 있다. latency와 RPS가 선형인지 확인한 뒤 capacity headroom을 판단한다.

예 2: `pool.hikariPending.max=8`, `hikariAcquireP95Ms=180`, `mysql.threadsRunning` 낮음

DB가 CPU로 포화됐다는 증거는 약하다. connection usage time과 transaction 내부 외부 I/O를 먼저 본다. 풀 크기부터 늘리면 원인을 숨길 수 있다.

예 3: `mysql.qps/RPS` 증가, `dbCpuUsPerQuery` 안정, `dbCpuMsPerReq` 증가

쿼리 한 건의 비용은 비슷하지만 요청당 쿼리 수가 늘었다. N+1 또는 반복 접근 가설이 강하다. repository method count와 SQL digest로 귀속한다.

15. 앞으로 수집하면 좋은 지표를 배우는 법

이 장의 목적은 구현 순서를 정하는 것이 아니라, 현재 지표가 답하지 못하는 질문과 새 지표가 메우는 빈칸을 이해하는 것이다. 새 지표는 많을수록 좋은 것이 아니다. 결론을 바꾸거나 다음 조사 행동을 분명하게 만드는 지표가 가치 있다.

15.1 가장 먼저 이해할 직접 실패 신호

후보	무엇을 직접 말하나	현재 지표와 연결
log.errorCount / warnCount	실행 구간에 앱 오류·경고 로그가 새로 발생	k6 error가 잡지 못한 내부 예외
gc.fullCount / fullPauseMaxMs	major/full GC가 실제 발생	합산 GC count/max의 모호함 제거
container_oom_events_total	컨테이너 OOM 사건	memory saturation의 결과 확인
container_memory_failcnt	메모리 limit 할당 실패	순간 peak·limit 충돌
TCP ListenDrops/Overflows	서버가 새 연결을 수락하지 못함	connecting 상승의 서버측 증거
Hikari timeout	connection 대기 후 요청 실패	pending/acquire의 실제 영향

이런 신호는 0이 정상인 경우가 많고 1건부터 조사를 시작할 수 있다. 다만 원시 누적값이 아니라 실행 구간 `increase()`를 저장해야 한다.

15.2 원인을 갈라 주는 자원 지표

후보	새로 답하는 질문	해석 포인트
Hikari usage avg/max/p95	connection을 왜 오래 잡았나?	acquire는 결과, usage는 원인 후보
Host PSI cpu/io/memory	작업이 자원 때문에 멈춘 총 시간은?	load average보다 직접적인 stall
disk read/write await	I/O 한 건이 얼마나 오래 걸렸나?	bytes와 latency 분리
disk queue depth	동시에 밀린 I/O가 몇 개인가?	await와 함께 장치 포화 판단
MySQL fsync/log waits	커밋 내구성 비용으로 기다렸나?	write-heavy·disk await와 정렬
dirty pages/wait free	buffer pool flush가 따라가지 못했나?	checkpoint·disk pressure
row lock avg/max	락 대기가 자주 짧나, 드물고 기나?	count/total의 모호함 제거

> **E-46 때문에 PSI 우선순위가 높다**
>
> 일반적인 프로젝트에서는 host PSI가 P2일 수 있다. 하지만 현재는 같은 조건의 반복 세션마다 효율이 크게 달라진 사례가 있어, 컨테이너 밖 host 경쟁·I/O stall을 확인할 직접 신호로 우선순위가 올라간다.

15.3 귀속 벡터: 어느 기능이 만들었나

후보	저장 단위	분석 가치	cardinality 통제
HTTP top-K	method·templated URI·status	느린 endpoint와 5xx 귀속	상위 N + 오류 URI
Repository top-K	repository·method	N+1·비싼 데이터 접근 귀속	count/total 기준 top-K
Redis command	command	KEYS·SET·LTRIM 비용 분리	remote address 제거
Scheduler	class·method	주기 spike 귀속	작업 수가 작아 전부 가능

현재 수집기는 많은 Prometheus 시계열을 하나의 스칼라로 접는다. 귀속 정보는 `infra.flat` 한 줄이 아니라 별도 배열 구조로 보존해야 한다. 무제한 label 조합을 저장하면 파일이 커지고 비교가 깨지므로 top-K와 안정된 label만 남긴다.

15.4 k6에서 빠진 클라이언트 증거

후보	답하는 질문	주의
dropped_iterations	목표 iteration을 만들지 못했나?	arrival-rate capacity 판단의 필수 전제
http_req_sending	요청 payload 전송이 느렸나?	업로드·클라이언트 NIC
http_req_receiving	응답 body 수신이 느렸나?	큰 payload를 server processing으로 오진 방지
measure phase breakdown	실제 측정 구간에서만 어디가 느렸나?	warmup/all 구간과 섞지 않기

15.5 SQL digest: 지표에서 코드로 가는 다리

`performance_schema.events_statements_summary_by_digest`는 정규화 SQL별 실행 횟수·총 시간·평균 시간·rows examined·rows sent·임시 테이블 정보를 준다. 실행 전 요약을 초기화하고 실행 후 top 20을 저장하면 '어느 SQL이 이번 실행의 시간을 소비했나'를 재현 가능하게 남길 수 있다.

패턴	digest 해석
count 매우 높음, avg 낮음, total 높음	N+1·과다 호출; 한 번은 싸지만 합계가 큼
count 낮음, avg/max 높음	드문 고비용 쿼리; tail latency 후보
rows examined ≫ rows sent	필터/인덱스 비효율
tmp disk table 증가	정렬·GROUP BY·메모리 임시테이블 문제

15.6 조건부 심층 자료

JFR, GC log, MySQL slow log, Redis SLOWLOG는 원인 귀속에 강하지만 파일이 크고 개인정보·SQL literal·보관 비용을 고려해야 한다. 기본 실행마다 무조건 저장하기보다 실패·WARN·명시 옵션 때 수집하고 최대 크기와 보존 기간을 둔다.

15.7 기본 지표로 넣지 말아야 할 것

- `messageBrokerTaskScheduler` queued 값을 batch backlog로 이름 붙이지 않는다.
- 모든 URI·status·exception label 조합을 영구 저장하지 않는다.
- raw error counter나 KEYS 누적 count를 실행 값으로 저장하지 않는다.
- client p95 - server p95를 '네트워크 p95'로 계산하지 않는다.
- 새 지표를 정상 분포 없이 즉시 regression gate로 승격하지 않는다.

16. 실전 연습문제 24선

먼저 숫자를 보고 관측 사실만 한 문장으로 적는다. 다음에 가설, 반증 지표, 다음 실험을 적는다. 답을 맞히는 것보다 단정과 증거를 분리하는 연습이 중요하다.

> **답안 형식**
>
> ① 사실 ② 가능한 가설 2개 ③ 함께 볼 지표 ④ 반증 조건 ⑤ 조건 하나만 바꾸는 실험

기초 문제

문제 1. avg·p95·max

CPU avg 40%, p95 44%, max 100%다. 'CPU가 지속 포화'라고 말할 수 있는가? 세 요약값이 각각 무엇을 말하는지 설명하라.

문제 2. Counter

`logback_events_total{level="error"}=56`을 봤다. 이번 테스트에서 오류가 56건이라고 결론 내리면 왜 안 되는가?

문제 3. CPU 포화

앱 CPU max가 1.6 cores이고 limit가 2 cores다. 포화도를 계산하고, 이것만으로 병목을 확정할 수 없는 이유를 쓰라.

문제 4. 메모리 층

heap used는 700MB로 안정적인데 container working set이 2.3GB까지 오른다. 어떤 메모리 영역을 의심할 수 있는가?

문제 5. QPS/RPS

RPS 100에서 MySQL QPS가 1,200이다. 요청당 쿼리 수를 계산하고 이 값이 곧 N+1 증명인지 판단하라.

문제 6. k6 waiting

k6 waiting p95가 500ms다. 이것을 서버 애플리케이션 처리시간 500ms라고 부르면 안 되는 이유는?

중급 문제

문제 7. 높은 CPU

CPU saturation 93%, throttle 0%, p95 안정, RPS는 이전보다 40% 높고 CPU/req는 동일하다. 가장 타당한 해석과 다음 행동은?

문제 8. 낮은 평균·높은 throttle

CPU avg 55%인데 throttledPct 7%, p99가 주기적으로 튄다. 어떻게 이런 조합이 가능한가? 어떤 시계열을 볼 것인가?

문제 9. Hikari queue

pending max 8, acquire p95 180ms, usage p95 900ms, MySQL CPU 35%, row lock wait 0이다. 원인 후보와 우선 실험을 쓰라.

문제 10. N+1 구분

변경 후 dbCpuMsPerReq는 2배, dbCpuUsPerQuery는 동일, queriesPerReq는 2배가 됐다. 무엇이 바뀐 것인가?

문제 11. Redis 캐시

Redis hit ratio가 97%에서 72%로 내려가고 MySQL QPS·CPU와 endpoint p95가 함께 올랐다. 어떤 인과 경로를 가정할 수 있고 무엇으로 확인할 것인가?

문제 12. GC max

GC pause max 420ms, p95 8ms, overhead 1%다. 서비스 p99가 600ms로 올랐다. GC가 원인이라고 확정할 수 있는가?

문제 13. Disk

disk bytes/sec는 낮지만 read await와 host iowait가 높다. 'I/O가 적으니 디스크 병목 아님'이 왜 틀릴 수 있는가?

문제 14. Scheduler

endpoint p99가 약 60초마다 튀고 ViewCountScheduler count와 KEYS count가 같은 비율로 증가한다.
사실·가설·증명을 분리해 쓰라.

문제 15. Load generator

목표 1,000 RPS인데 실제 720 RPS에서 평평하고 dropped_iterations와 loadgen throttling이 증가한다. 서버 capacity는 720 RPS인가?

문제 16. 오류 로그

새 앱 인스턴스에서는 error counter가 0, 오래된 인스턴스에서는 56이다. 실행 비교에 사용할 올바른 값과 query 형태를 설명하라.

고급 문제

문제 17. CV와 추세

08-16 세트는 Spearman +0.90, 전체 +9%, CV 3.12%다. CV가 작으니 4% 개선이면 진짜라고 말하면 왜 안 되는가?

문제 18. ABBA

환경이 실행마다 느려지는 상황에서 후보 A와 B를 비교한다. A,A,A,A,B,B,B,B 대신 ABBA/BAAB 순서를 쓰는 이유를 설명하라.

문제 19. 분위수 차감

client p95 700ms, server p95 400ms다. 네트워크 p95=300ms라고 계산할 수 없는 통계적 이유와 시스템적 이유를 각각 쓰라.

문제 20. 가짜 포화

memory saturation이 227%로 계산됐다. 이것을 OOM 위험이라고 보고할 것이 아니라 측정 오류를 먼저 의심해야 하는 이유와 가능한 원인을 쓰라.

문제 21. Pool tuning

Tomcat 400, Hikari 10에서 pending이 생겼다. Hikari를 40으로 늘리는 실험의 이점·위험·성공 판정 지표를 설계하라.

문제 22. E-46

같은 dataset·seriesHash인데 세션 A→C에서 p95 +45%, RPS -25%다. 원인을 좁히는 측정 설계를 제시하라.

문제 23. Redis KEYS 인과

KEYS가 60초마다 호출되는 사실만으로 사용자 지연의 원인이라고 할 수 없다. 원인임을 지지하거나 반박할 대조 실험을 설계하라.

문제 24. 종합 진단

p99↑, RPS↓, app CPU 낮음, Hikari pending↑, usage↑, MySQL row lock time↑, Redis 정상이다. 병목 전파 경로와 코드 수준 조사 순서를 쓰라.

17. 연습문제 해설

문제 1 해설

지속 포화라고 말할 수 없다. avg 40%는 전체적으로 낮고 p95 44%는 시간의 95%가 낮았다는 뜻이다. max 100%는 매우 짧은 단발 spike 한 번일 수 있다. 원시 time series와 spike 시각의 latency·scheduler를 확인한다.

문제 2 해설

56은 프로세스 시작 이후 누적값일 수 있다. 실행 구간의 새 오류는 `increase(logback_events_total{level="error"}[$RANGE])`로 구한다. 재시작·인스턴스 label 범위와 요청 수 대비 오류율도 확인한다.

문제 3 해설

1.6/2×100=80%다. 80%는 여유가 줄었다는 사실일 뿐 대기나 사용자 영향이 있다는 증거는 아니다. throttle, p95/p99, RPS, CPU/req가 함께 움직이는지 본다.

문제 4 해설

heap 밖의 Metaspace, Code Cache, thread stack, direct buffer, native library, mmap/page cache를 의심한다. RSS·nonheap·thread count·direct buffer와 OOM/failcnt를 확인한다.

문제 5 해설

1,200/100=요청당 12 queries다. 업무상 12개가 정상일 수 있어 N+1 증명은 아니다. 같은 endpoint·dataset에서 N을 늘릴 때 query count가 비례하는지, repository method와 SQL digest count를 확인한다.

문제 6 해설

waiting은 요청을 보낸 뒤 첫 바이트까지의 클라이언트 관점으로, 서버 accept/queue, Tomcat 대기, 앱 로직, DB/Redis, 네트워크 일부가 섞인다. endpoint별 server timer와 같은 요청의 trace가 필요하다.

문제 7 해설

부하가 40% 늘면서 CPU도 비례해 오른 정상 scaling일 가능성이 가장 크다. 작업당 비용과 latency가 안정적이고 throttle이 없기 때문이다. 여유 7%가 운영 spike를 견딜지 capacity headroom을 평가하되 코드 회귀로 단정하지 않는다.

문제 8 해설

CFS quota는 짧은 주기별 제한이라 burst가 허용량을 일찍 소진하면 나머지 주기에 멈춘다. 1분 평균은 이를 낮게 보일 수 있다. 1~5초 CPU cores, throttled periods/seconds, p99 spike, scheduler 실행 시각을 겹친다.

문제 9 해설

DB CPU와 row lock이 낮으므로 DB 계산·락보다 connection을 오래 보유하는 앱 경계를 먼저 의심한다. usage p95가 900ms인 endpoint·transaction을 찾고, transaction 안의 S3/Redis/직렬화·긴 루프를 밖으로 옮긴 대조 실험을 한다. pool 확대는 2차 실험이다.

문제 10 해설

쿼리 한 건의 CPU 비용은 그대로인데 요청 하나가 만드는 쿼리 수가 2배가 됐다. 반복 쿼리 또는 N+1이 가장 강한 가설이다. repository count와 digest count를 변경 전후 비교한다.

문제 11 해설

cache miss 증가가 DB 조회 증가로 이어져 MySQL CPU와 endpoint 지연을 높였다는 경로다. endpoint별 cache hit/miss, Redis command count, 동일 요청의 DB query count를 확인하고 cache를 warm/disable한 대조 실험을 한다.

문제 12 해설

확정할 수 없다. max 420ms 한 번은 p95와 overhead에 거의 안 보일 수 있고 p99와 시간적으로 겹쳤을 수 있지만, 우연히 겹친 다른 queue도 가능하다. GC event timestamp와 전체 endpoint 동시 spike, DB/pool/network를 겹쳐 본다.

문제 13 해설

작은 랜덤 read나 fsync는 bytes는 적어도 각 요청의 latency가 길 수 있다. await·queue·PSI io와 MySQL pending fsync/log waits를 확인한다. 대역폭과 응답시간은 별도 축이다.

문제 14 해설

사실은 scheduler 실행과 KEYS 호출이 약 60초 주기로 함께 발생하고 endpoint p99도 비슷한 주기로 튄다는 것이다. 가설은 KEYS 또는 sync 작업이 Redis/DB를 막는다는 것. command completion, Redis CPU, scheduler duration을 시간 정렬하고 SCAN/no-op 대조로 p99가 사라지는지 증명한다.

문제 15 해설

아니다. 발생기가 목표 부하를 만들지 못했으므로 서버 한계를 관측하지 못했을 수 있다. loadgen CPU limit을 늘리거나 발생기를 분리하고 dropped=0인 조건에서 다시 올린다. 서버 saturation도 함께 확인한다.

문제 16 해설

두 인스턴스의 raw 누적값은 시작 시점이 달라 비교 불가다. 각 실행 창의 increase를 같은 job/instance 범위로 합치고, 필요하면 1,000 요청당 error log 수로 정규화한다.

문제 17 해설

CV는 값의 흩어짐만 재고 순서 방향을 제거하지 않는다. +0.90의 강한 단조 증가와 +9% drift가 있으므로 표본은 동일한 중심의 무작위 noise가 아니다. 4% 차이는 실행 순서만으로도 생길 수 있어 기존 CV/MDE로 진위를 판정할 수 없다.

문제 18 해설

몰아서 측정하면 처리 A/B가 시간과 완전히 겹친다. 교차 순서는 각 처리가 앞·뒤 시간을 나눠 가져 선형 drift를 상쇄한다. 시간 block 안의 paired A-B 차이를 비교하고 순서를 무작위화한다.

문제 19 해설

통계적으로 서로 다른 요청 집합의 95분위는 같은 요청의 구성요소가 아니므로 분위수끼리 뺄 수 없다. 시스템적으로 client에는 serialization, queue, transport가 섞이고 server timer의 시작/끝 범위도 다르다. trace 또는 동일 요청의 단계별 timer가 필요하다.

문제 20 해설

물리적으로 cgroup 사용량은 상한의 227%로 지속할 수 없고 강제 회수/OOM이 일어나야 한다. 같은 이름의 이전·현재 컨테이너 시계열 합산, 분자·분모 label 범위 불일치, scrape 시점 불일치를 의심한다. 현재 도구는 105% 초과를 결측 처리한다.

문제 21 해설

이점은 connection queue 감소 가능성이다. 위험은 DB 동시 쿼리 증가로 CPU·I/O·lock이 악화되는 것이다. 같은 부하에서 pending/acquire/timeout, usage, MySQL CPU/threadsRunning/lock, p95와 처리량을 함께 비교한다. transaction diet 실험도 별도로 둔다.

문제 22 해설

같은 declared 조건만 믿지 말고 warmup·실행 순서·host PSI·iowait/steal·loadgen throttle·JIT/GC·scheduler·cache 상태를 실행별 저장한다. ABBA block과 container restart/cooldown 조건을 나눠 재현하고, 시간 trend를 포함한 paired 비교를 한다.

문제 23 해설

기준 실행과 동일 keyspace에서 (A) KEYS 구현, (B) cursor SCAN 구현 또는 해당 scheduler no-op을 교차 실행한다. KEYS completion·Redis CPU·다른 command p95·endpoint p99를 시간 정렬한다. B에서 주기 spike가 일관되게 사라지면 인과가 강해진다.

문제 24 해설

DB row lock 대기가 transaction을 늘리고 connection usage가 길어져 Hikari pending이 생기며 Tomcat 요청이 기다려 p99 상승·RPS 감소로 전파된 경로가 가장 타당하다. lock을 만드는 SQL/digest와 transaction 범위를 찾고, 동일 row hot-key·scrap/reaction concurrency를 확인한다. Redis와 app CPU는 우선순위가 낮다.

부록 A. 증상별 1페이지 진단표

보이는 증상	첫 확인	강한 동반 증거	반증·다음 실험
p99만 상승	max/time series	GC max, pending, scheduler	시간 정렬; 전체 p95만 보지 않기
RPS plateau	dropped·loadgen	서버 saturation·queue	발생기 여유 확보 후 재시험
CPU 상승	cores/limit/throttle	CPU/req·JFR	부하 증가인지 비용 증가인지 분리
메모리 상승	working set/limit	heap/live data/OOM	JVM 내부·native 분리
DB 지연	Hikari pending/usage	MySQL CPU/lock/disk	digest·transaction 경계
Redis 의심	command completion	hit/eviction/CPU	명령 대조·SLOWLOG
주기 spike	scheduler/GC/flush	동일 주기 상관	작업 no-op·주기 이동 실험
세션 drift	Spearman/time	PSI·JIT·cache·열	ABBA block·조건 고정

분석할 때 말로 읽는 순서

'사용자 p99가 올랐다. 부하 발생기는 dropped와 throttle이 없어 목표 부하를 만들었다. 앱 CPU는 낮고 Hikari pending·usage가 올랐으며 같은 시각 MySQL row lock time이 증가했다. 따라서 DB lock → connection 보유 증가 → pool queue → 사용자 tail latency 경로를 우선 가설로 둔다. digest와 transaction을 확인하고 hot-row를 분산한 실험으로 검증한다.'

부록 B. 현재 임계값 빠른 참조

아래는 `performance/regression/rules.json`과 현재 도구의 운영 판단을 학습용으로 요약한 것이다. 서비스 SLO·시나리오·정상 분포에 맞춰 검증해야 하며 보편 기준이 아니다.

영역	WARN	FAIL	주의
k6 p95	상대 규칙	> 500ms	시나리오별 목표가 달라야 함
k6 p99	상대 규칙	> 1200ms	timeout 검열 확인
k6 error	> 0.1%	> 1%	오류 종류 분해
CPU saturation	> 80%	> 95%	throttle·latency 동반
CPU throttle	> 1%	> 10%	loadgen FAIL은 >5%
Memory saturation	> 85%	> 95%	OOM/failcnt가 직접 신호
GC pause max	> 200ms	> 500ms	SLO와 시간 정렬
GC overhead	> 5%	> 10%	할당·live data 확인
MySQL buffer hit	< 99%	< 95%	읽기 표본 수·disk 동반
Aborted connects	> 0	> 10	구간 increase
Redis hit	< 90%	-	DB 부하 전파 확인
Redis eviction	> 0	> 1000	memory/max와 함께
Hikari saturation	> 80%	> 95%	pending·usage
Hikari pending	> 0	> 5	지속 시간 중요
Hikari acquire p95	> 10ms	> 100ms	timeout 검열
Hikari timeout	-	> 0	직접 실패
Tomcat saturation	> 70%	> 90%	DB queue와 구분

부록 C. 단위·공식·용어

단위와 공식

항목	공식·변환
bytes → MiB	bytes / 1024^2
bytes → GiB	bytes / 1024^3
seconds → ms	seconds × 1000
CPU cores	rate(cpu_usage_seconds_total)
평균 timer	rate(sum) / rate(count)
구간 사건 수	increase(counter[$RANGE])
초당 속도	rate(counter[$RANGE])
포화도	사용량 / 한계 × 100
요청당 비용	구간 총비용 / 구간 완료 요청 수
CV	표준편차 / 평균 × 100

핵심 용어

용어	쉬운 뜻
Bottleneck	수요 증가가 queue·지연·오류로 바뀌는 제한 지점
Saturation	사용량이 물리·설정 한계에 가까운 정도
Tail latency	느린 쪽 끝의 지연, 보통 p95/p99
Throughput	단위 시간에 완료한 작업량
Backpressure	뒤 단계가 느려 앞 단계가 속도를 줄이거나 대기하는 현상
Cardinality	label 조합으로 생기는 시계열 수
Censoring	timeout 상한 때문에 실제로 더 느린 값을 정확히 못 보는 현상
Attribution	총비용을 endpoint·SQL·command·job에 귀속하는 것
Confounder	코드 효과와 함께 변해 원인 판단을 흐리는 제3의 요인
Warmup	JIT·cache·connection 등이 안정 상태에 도달하는 준비 구간

부록 D. 4주 학습 루틴

주차	학습	직접 해볼 일	통과 기준
1주	1~5장	기존 run 3개의 CPU·memory·disk를 사실 문장으로 요약	단정 없이 단위·범위 설명
2주	6~11장	한 run의 HTTP→pool→MySQL/Redis 경로를 시간 순서로 설명	동반 지표와 반증 제시
3주	12~15장	BTL-002·005·007·KI-17을 지표 관점에서 다시 진단	직접/간접 증거 구분
4주	16~17장	24문제를 답 없이 다시 풀고 실제 run 하나에 진단 보고서 작성	실험 하나를 명확히 설계

최종 자기 점검

- avg/max/p95가 서로 다른 표본 질문임을 설명할 수 있다.
- CPU 사용량·포화·throttle·요청당 비용을 분리할 수 있다.
- heap/RSS/working set과 OOM 직접 신호를 연결할 수 있다.
- Hikari acquire와 usage, MySQL connection/lock을 연결할 수 있다.
- k6 client 시간과 server timer를 함부로 빼지 않는다.
- 귀속 지표가 없을 때 원인을 확정하지 않고 필요한 증거를 말할 수 있다.
- 추세가 있는 반복 측정에서 CV/MDE를 noise 기준으로 사용하지 않는다.
- ABBA·block·warmup·dataset 고정으로 비교 실험을 설계할 수 있다.

부록 E. 이 교재의 프로젝트 근거

이 PDF는 2026-08-18 현재 로컬 워킹트리를 기준으로 작성했다. 지표 이름·개수는 생성 시 `metrics-catalog.js`를 직접 읽었다. 아래 문서는 프로젝트 사례와 판단 기준의 근거다.

근거 파일	사용한 내용
performance/tools/lib/metrics-catalog.js	현재 카탈로그 키·단위·설명·파생 포화도
localDocs/perf-metrics-catalog-plan.md	현재 공백, 수집 후보, 적용/조건부/제외 판단
localDocs/perf-session-drift.md	세션 drift, 반복 순서 추세, 비교 가능성
performance/reports/repeatability/*.json	08-16·08-17 반복 측정 요약
performance/bottlenecks/BTL-002	Tomcat-Hikari 구조적 불균형
performance/bottlenecks/BTL-005	댓글 N+1 가설과 직접 검증
performance/bottlenecks/BTL-007	scheduler burst 가설
docs/KNOWN-ISSUES.md	Redis KEYS, transaction 외부 I/O, 댓글 반복 쿼리
environment/docker-compose.perf.yml	앱·MySQL·Redis CPU/메모리 limit
application.properties / application-perf.properties	Tomcat·histogram·JVM 관련 설정

> **읽은 뒤의 원칙**
>
> 지표는 결론이 아니라 관측창이다. 좋은 분석은 숫자를 많이 외우는 능력이 아니라, 값의 범위와 단위를 확인하고 여러 계층의 시간 관계를 연결하며 반증 가능한 실험을 만드는 능력이다.

끝